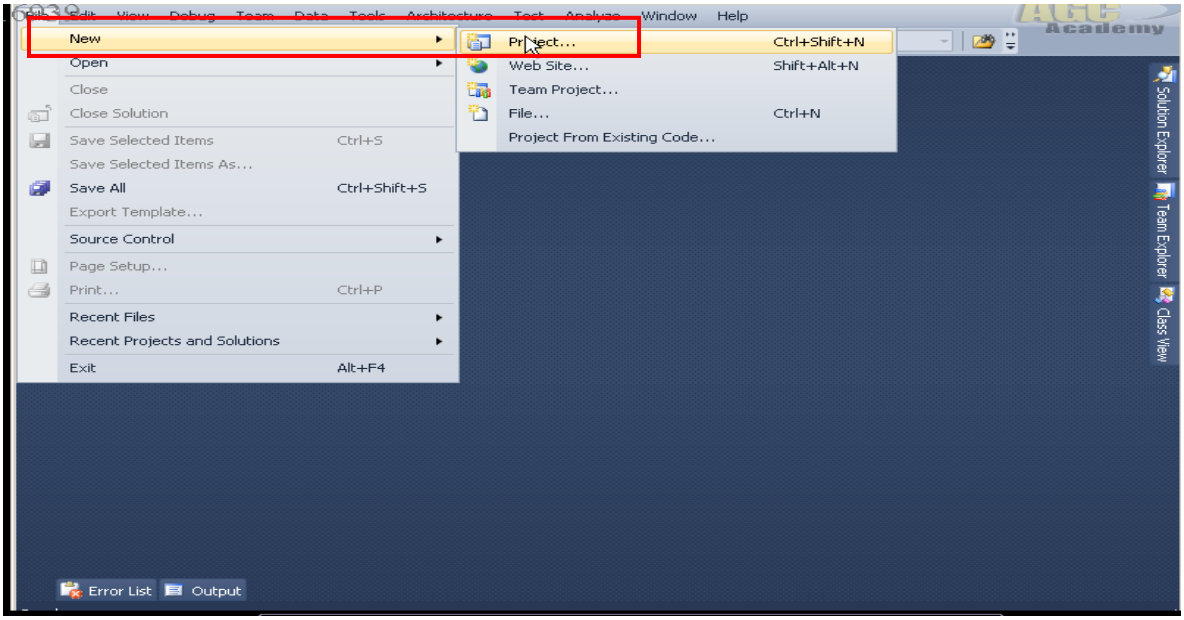
بسم اللّه الرحمن الرحيم

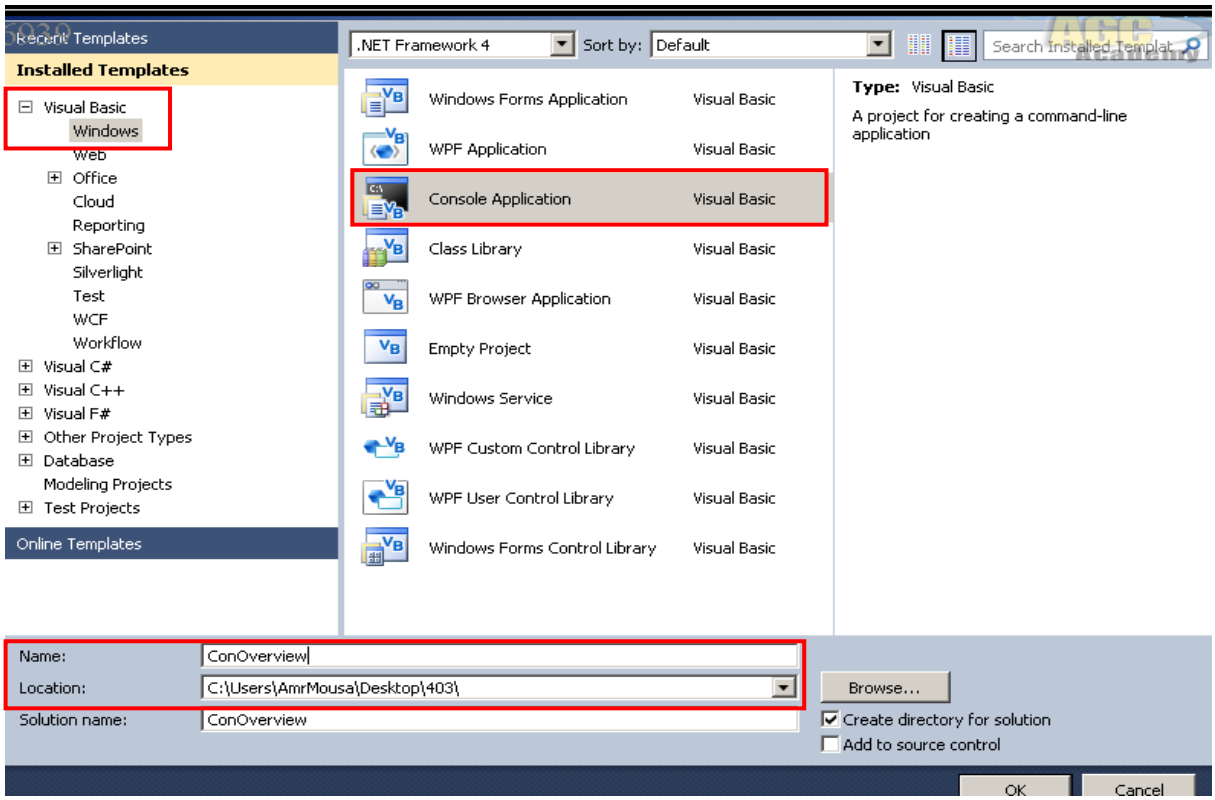
البحث الخاص بالكور ٤٠٣

١- لإنشاء مشروع يعمل تحت بيئة سطر الاوامر Command Line نذهب الي

New→ Project

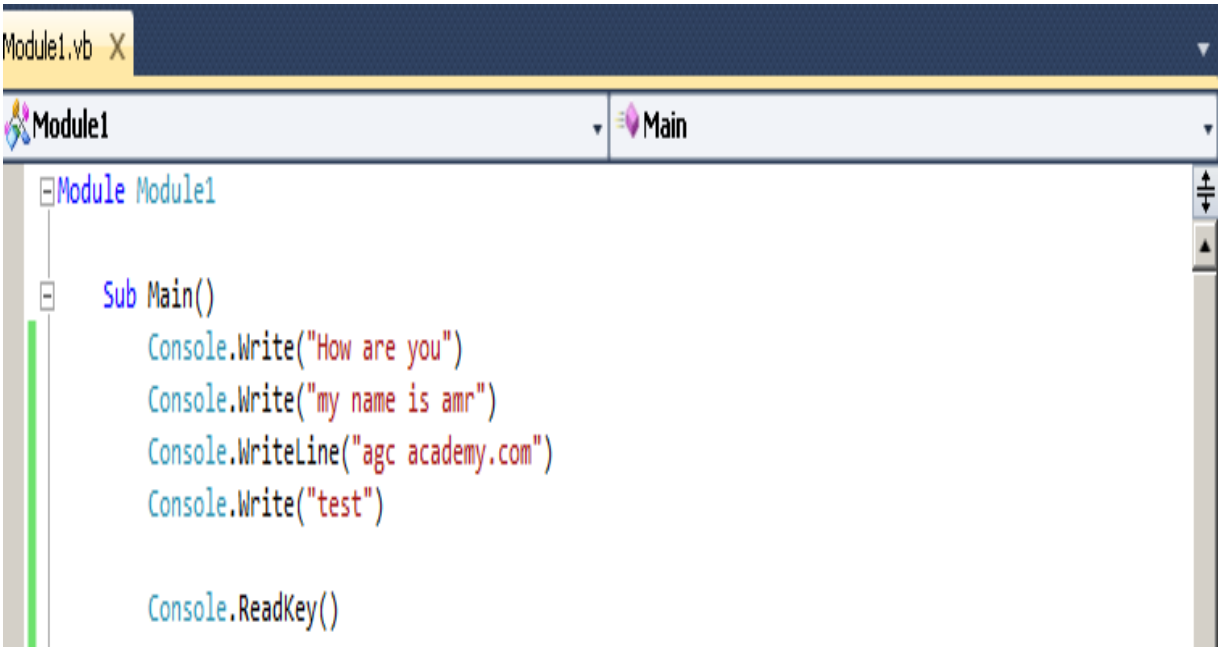


ثم نختار من القائمة اليسرى نوع لغة البرمجة ثم نختار من القائمة الوسطي نوعية البرنامج وسيكون في هذه الحالة Console Application ومن اسفل الشاشة نختار المشروع والمكان الذى سيتم حفظ المشروع بداخله من كما بالشكل التالي:

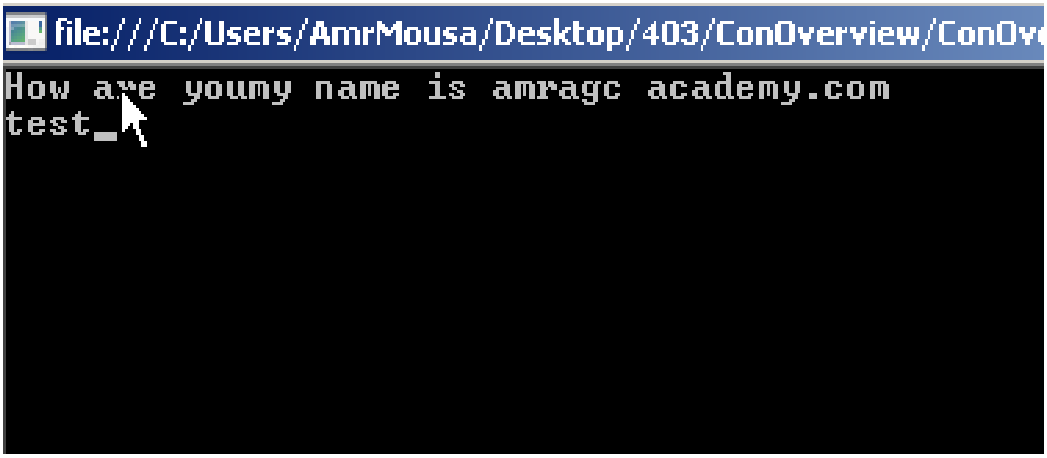


بعد انشاء المشروع يبدأ بإنشاء Module1 وهو في هذه الحالة بعد ذلك يقوم بإنشاء البرنامج الفرعي الرئيسي وهو يكون بالاسم Main وهذا البرنامج هو اول ما يتم تنفيذه بداخل الموديل ولابد من تواجد برنامج فرعي واحد بالاسم Main بداخل الموديل ولا يمكن تكرار نفس الاسم وعن طريق Main نقوم باستدعاء باقي البرامج الفرعية التي يتم كتابتها بداخل الموديل.

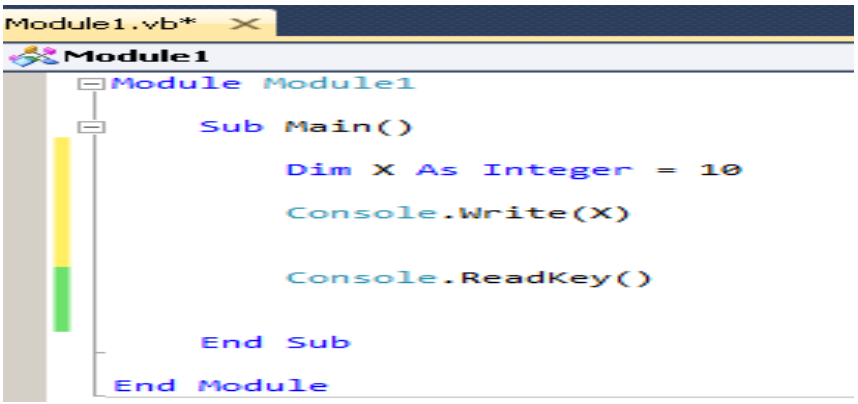
الاوامر التالية هي الاوامر الخاصة بإرسال قيم لتظهر علي شاشة الطرفية او سطر الاوامر او في بعض الاحيان يطلق عليه شاشة الدوس والامر الاول في الشاشة التالية هو الامر Console.Write("How are you") ويتسبب هذا الامر في طباعة عبارة How are you علي الشاشة وبالمثل الامر Console.WriteLine مسئول عن طباعة العبارة التي نريدها ويقوم بعد ذلك بنقل المؤشر الي اول السطر التالي وليس الي نفس السطر الحالي، والامر التالي في الصورة التالية هو الامر Console.ReadKey() وهذا الامر مسئول عن استقبال حرف من المستخدم فهو يقوم بالانتظار حتى يقوم المستخدم بالضغط علي اي مفتاح ثم يقوم بالخروج من البرنامج.



وعند تنفيذ الاوامر السابقة تظهر في الشاشة كما بالشكل التالي وبعد ان يتم طباعة الجمل العبارات الاربع السابقة ينتظر البرنامج ان يقوم المستخدم بالضغط علي أي مفتاح وبعد ذلك يتم الخروج من البرنامج



هنا نقوم بإرسال قيمة متغير ليتم طباعتها



في حالة الرغبة في طباعة قيمة المتغير ولكن بعد ان يتم طباعة نص في هذه الحالة لا يمكن استعمال علامة الجمع + بسبب ان علامة الجمع تستخدم مع القيم الرقمية فقط ولذلك الامر التالي ينتج عنه رسالة خطأ بسبب محاولة طباعة قيمة رقيمة مع نص

```
        Dim X As Integer = 10

        Console.Write("AGC " + X)


        Console.ReadKey()

    End Sub
```

والطريقة الصحيحة لطباعة القيمة الرقمية مع النص هي استخدام ما يطلق عليه Format Specification في لغة البرمجة سي او محددات الصياغة وهي في الفيجوال بيزك عن طريق ارسال قيمة رقيمة بداخل النص الذى نود طباعته وعند الطباعة يتم استبدال تلك القيمة الرقيمة بالمتغير الذى يلي النص في قائمة المعاملات فمثلاً في الصورة التالية يتم استبدال الرقم صفر بداخل النص الي القيمة الموجودة بداخل المتغير X وهي في هذه الحالة ١٠

```
    Sub Main()
        Dim X As Integer = 10
        Console.Write("AGC {0} ", X)

        Console.ReadKey()

    End Sub
End Module
```

وهنا ايضا يتم استبدال الارقام ٠ و ١ و ٢ بالمتغيرات x , y , z ويتم اختيار القيمة التي يتم طباعتها علي حسب ترتيب المتغيرات

```
Sub Main()

    Dim X As Integer = 10
    Dim y As Integer = 200
    Dim z As Integer = 300

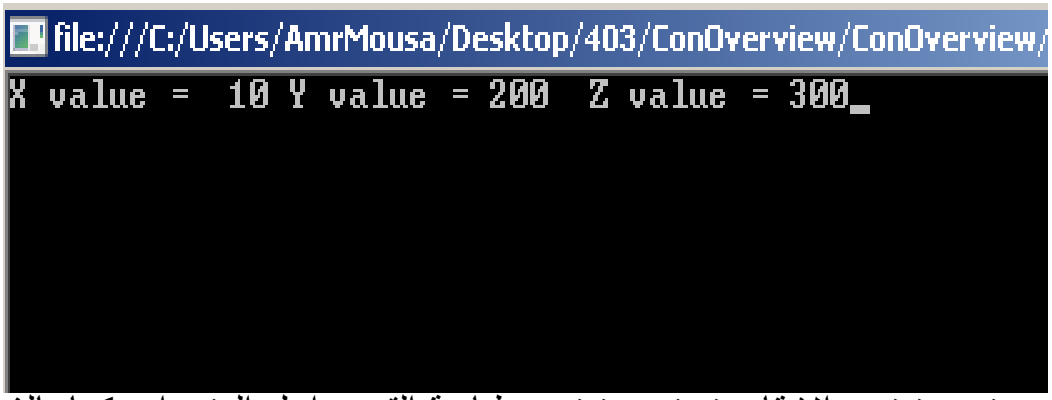

    Console.Write("X value =  {0} Y value = {1}  Z value = {2}", X, y, z)

    Console.ReadKey()


End Sub

End Module
```

وتظهر القيمة عند الطباعة بالشكل في الصورة التالي:



وعند تغير ترتيب الارقام يتم تغير ترتيب طباعة القيم بداخل المتغيرات كما بالشكل التالي:

```
Module Module1

    Sub Main()

        Dim X As Integer = 100
        Dim y As Integer = 200
        Dim z As Integer = 300
        Dim H As Integer = 400

        Console.Write("X value =  {2} {2} Y value = {1}  Z value = {0}", X, H, y, z)


        Console

    End Sub

End Module
```

```
file:///C:/Users/AmrMousa/Desktop/403/ConOverview/ConOverview/bin/Debug/ConOv
X value =  200 200 Y value = 400  Z value = 100_
```

ومن الممكن ايضاً طباعة أي قيم من متغيرات اخري مثل القيم النصية

```
Module Module1

    Sub Main()

        Dim username As String = "amrmousa"
        Dim EMail As String = "amrmousa@agcit.net"
        Dim Mobile As String = "0106675159"

                                                    opkarim116939
        Console.Write("User Name: {0}  Phone: {2}  my Email: {1}", username, EMail, Mobile)


        Console.ReadKey()


    End Sub

End Module
```

٢- الامر الاول خاص بتغير لون خلفية سطر الاوامر الي اللون الازرق الغامق

الامر الثاني خاص بعمل مسح للكتابة الموجودة في الشاشة.

السطر الثالث خاص بتغير حجم الشاشة اي الابعاد الخاصة بها.

الامر الرابع خاص بتغير عنوان الشاشة الي Agc Academy

```
Module1.vb
Module1                                    Main
Module Module1

    Sub Main()

        Console.BackgroundColor = ConsoleColor.DarkBlue
        Console.Clear()
        Console.SetWindowSize(40, 10)
        Console.Title = "AGC Academy"

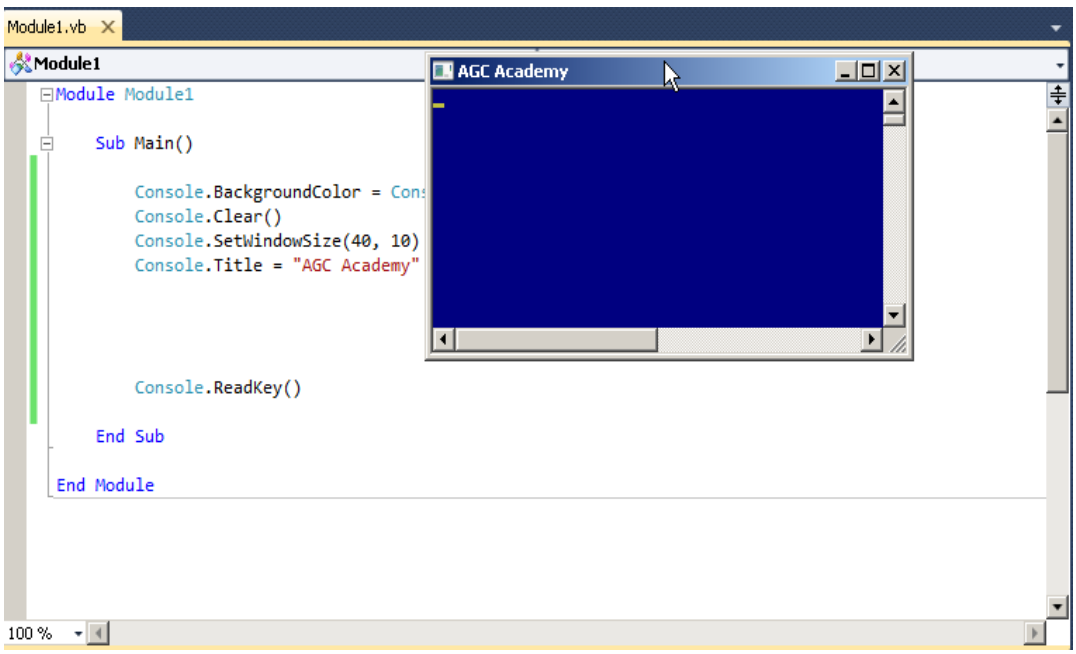


        Console.ReadKey()

    End Sub

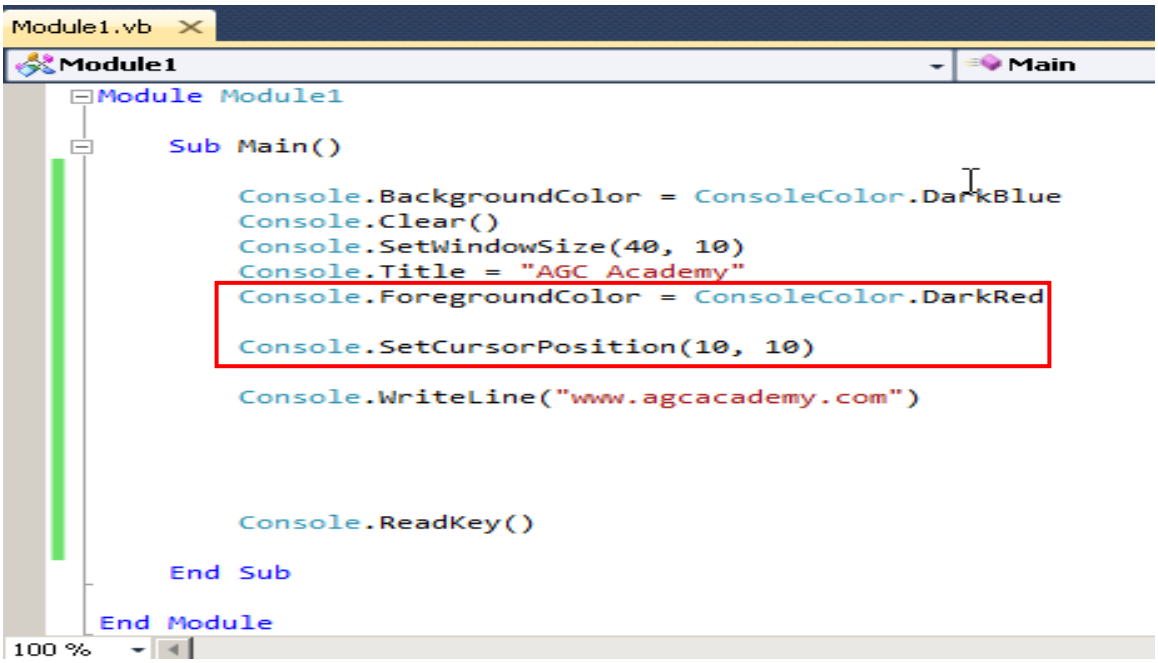
End Module
```

وعند التنفيذ تظهر الشاشة بالشكل التالي:

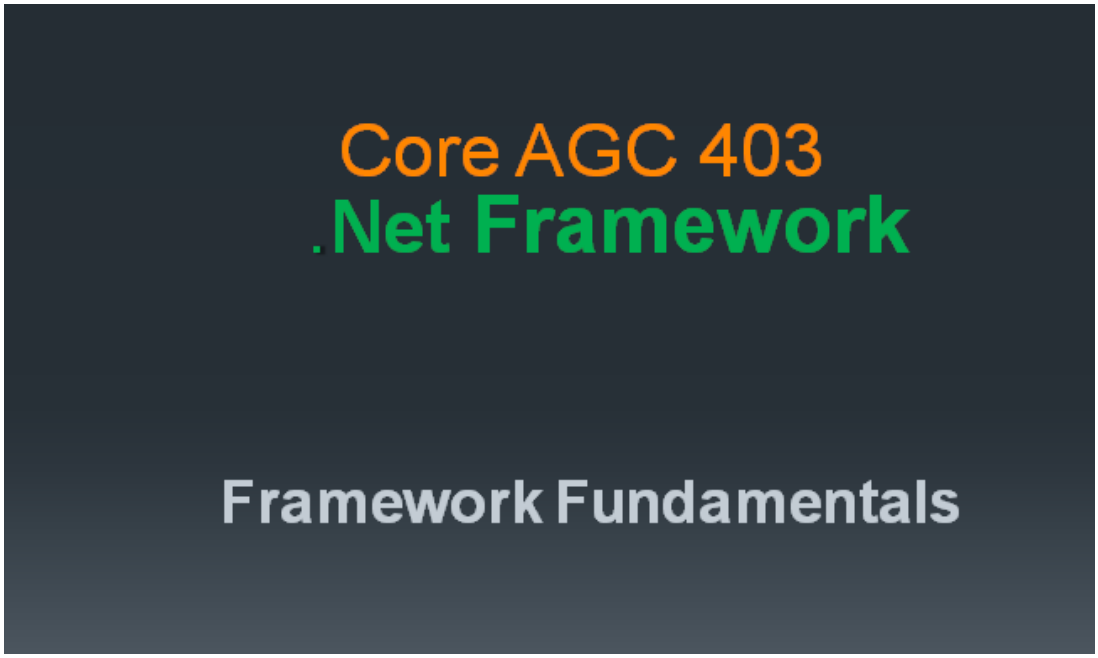


الامر الخامس خاص بتغير لون الكتابة التي يتم طباعتها او كتابتها في الشاشة.

الامر السادس خاص بتغير موقع مؤشر الذى يبدأ البرنامج التنفيذ والكتابة فيه.



٣- اساسيات اطار الدوت نت

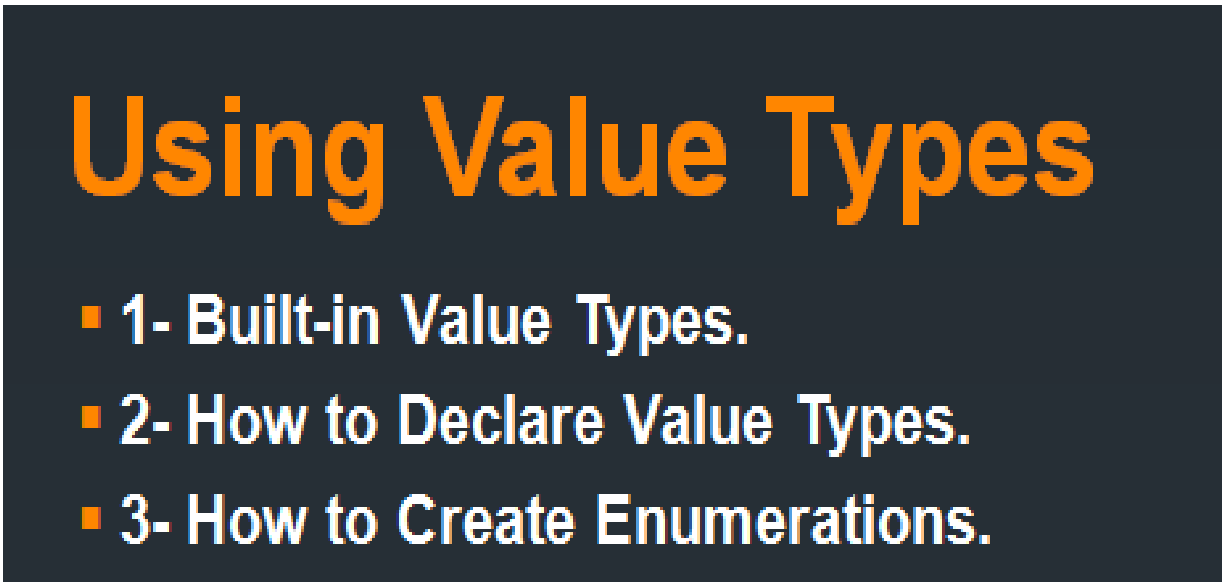


انواع القيم الموجودة بداخل اطار الدوت نت وسيتم التطرق الي :

١- انواع القيم الموجودة بداخل الدوت نت او بمعني اخر انواع المتغيرات.

٢- كيفية التصريح عن انواع المتغيرات.

٣- كيفية الاعلان عن Enumerations.



1- Built-in Value Types.

انواع المتغيرات او البيانات الموجودة بداخل الدوت نت في الشكل التالي:

النوع	القيمة	يستخدم في
Byte	0 to 255	وحدة ثنائية
Int16 Short	-32768 to 32767	عدد قصير
System.Int32 (Integer/int)	-2147483648 to 2147483647	عدد صحيح
System.UInt32 (UInteger/uint)	0 to 4294967295	عدد صحيح موجب
System.Int64 (Long/long)	-9223372036854775808 to 9223372036854775807	عدد طويل
System.Double (Double/double)	-1.79769313486232E+308 to 1.79769313486232E+308	عدد مزدوج
System.Decimal (Decimal/decimal)	-79228162514264337593543950335 to 79228162514264337593543950335	عدد عشري
System.Single (Single/float)	-3.402823E+38 to 3.402823E+38	عدد مفرد
Boolean	True or False / Yes or No	منطقي
Char	-	حرف
String		نصي
System.DateTime (Date/date)	1/1/0001 12:00:00 AM to 12/31/9999 11:59:59 PM	تاريخ ووقت
Object	-	عام ويتعامل مع جميع المتغيرات والقيم

2- How to Declare Value Types.

يتم التصريح عن المتغيرات بالطريقة العادية باستخدام الامر dim فمثلاً في الصورة التالية يتم الاعلان عن متغير من النوع Boolean ويتم طباعة التغير وهنا توجد ملاحظة ان القيمة الافتراضية للمتغير المنطقي هي False وذلك يعني انه عند الاعلان عن المتغير او التصريح عنه فانه يأخذ قيمة افتراضية هي False سواء قمت بعمل تخصيص لقيمة بداخله او لم اقم وبالمثل المتغير الرقمي integer يأخذ قيمة افتراضية وهي صفر.

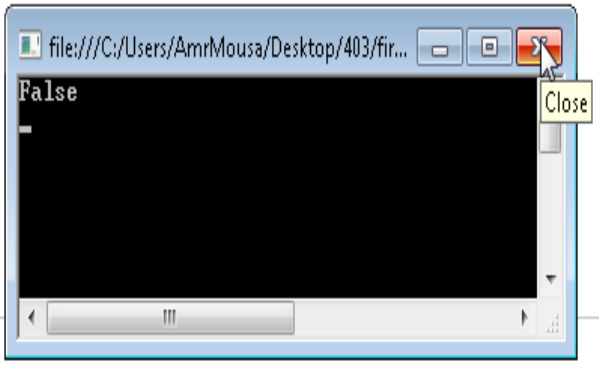
```
Sub Main()

    Dim B As Boolean
    Console.WriteLine(B)


    Console.ReadKey()

End Sub

End Module
```



وحين الرغبة في الاعلان عن متغير والتأكد من انه لا يأخذ قيمة بداخله نقوم باستخدام الدالة Nullable ونقوم بإعطائه قيمة Nothing كما بالشكل التالي:

```
Sub Main()
    Dim B As Nullable(Of Boolean) = Nothing
    Console.WriteLine(B)

    Console.ReadKey()
End Sub
End Module
```

ففي هذه الحالة تكون الدالة مسئولة عن اعطاء المتغير قيمة فارغة ففي هذا المثال المتغير B لا يحمل قيمة لا هو True أو False

وممكن بعد ذلك اعطاءه قيمة بشكل طبيعي

```
Module Module1

    Sub Main()

        Dim B As Nullable(Of Boolean) = Nothing
        B = True
        Console.WriteLine(B)

        Console.ReadKey()

    End Sub

End Module
```

فمثلاً الكود التالي يتحقق اذا كان بداخل المتغير قيمة ام لا وذلك باستخدام الدالة HasValue()

Module1.vb

Module1

```
Module Module1

    Sub Main()

        Dim B As Nullable(Of Boolean) = Nothing

        B = True

        If B.HasValue Then
            Console.WriteLine("HasValue")

        Else

            Console.WriteLine("Nullable value")

        End If


        Console.ReadKey()

    End Sub

End Module
```

100 %

Error List | Output

٤- من الممكن تعريف المتغيرات واعطاءها قيمة في نفس السطر

```
Module1                                          Main
Module Module1

    Sub Main()

        Dim X As String = "AGC Academy"
        Dim Url As String = "www.agcit.com"

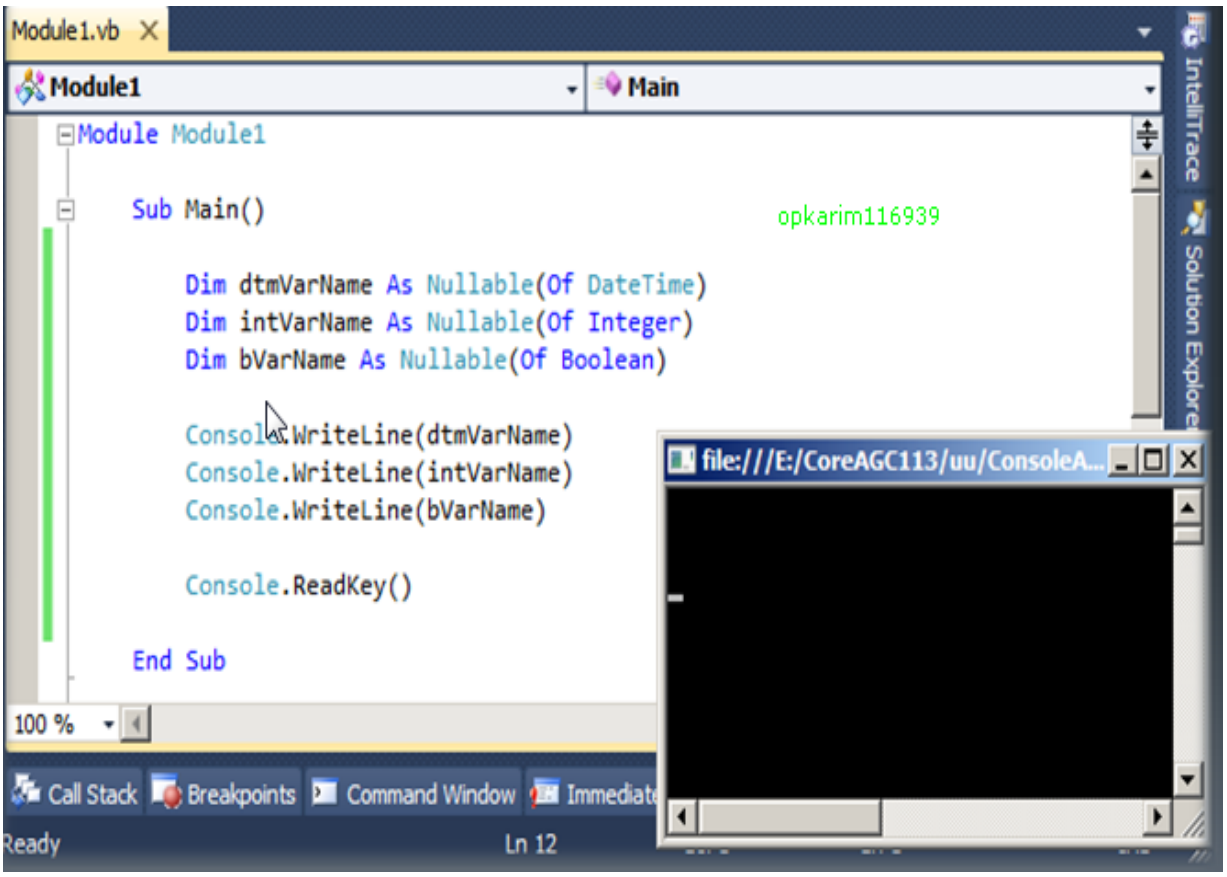


        Console.WriteLine("Company Name: " + X + " URL :" + Url)
        Console.ReadKey()

    End Sub

End Module
```

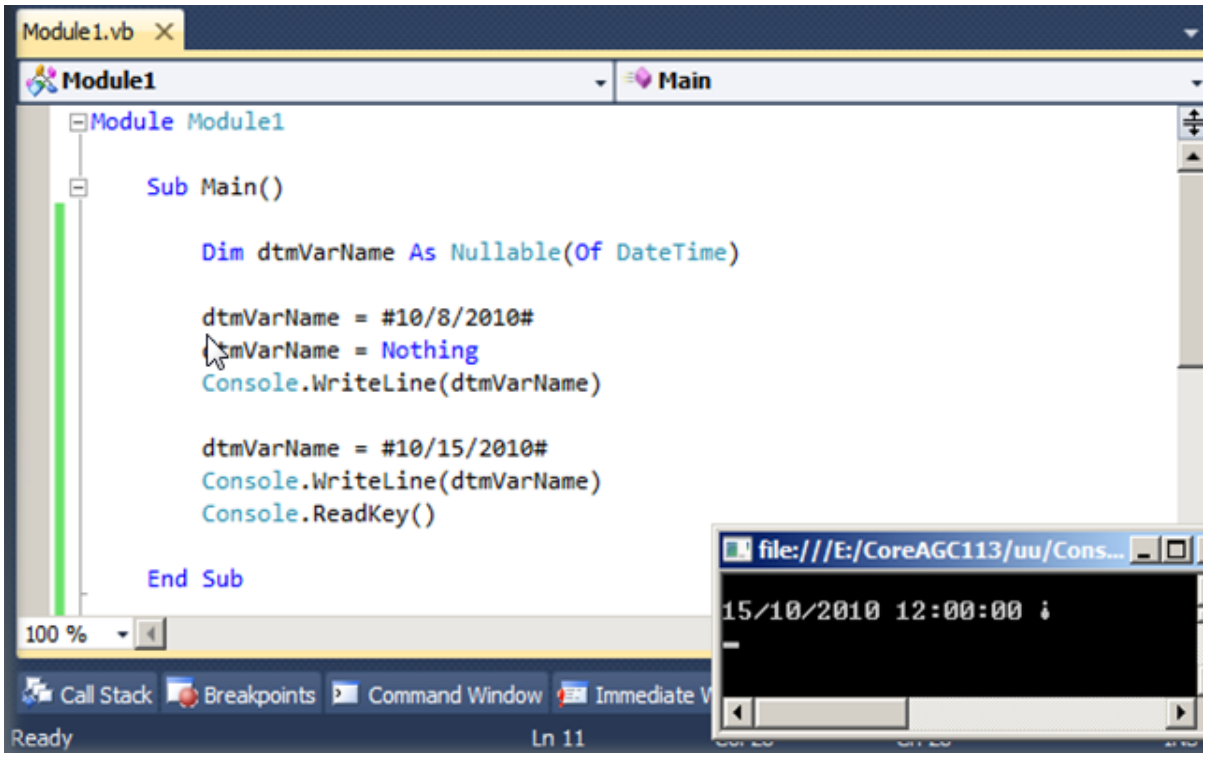
عند الاعلان عن المتغيرات من الممكن التأكد من عدم ادخال أي قيمة بداخل المتغيرات عن طريق الدالة Nullable واذا قمنا بطباعة المتغير الي الشاشة سيظهر فراغ في مكان المتغير كم في الصورة التالية:



يفضل عند الاعلان عن متغير ان يكون اسم المتغير يدل علي القيمة التي ستدخل الي المتغير ويفضل ان يكون الاحرف الاولي في اسم المتغير ترمز الي نوعية البيانات التي ستدخل في المتغير.

فمثلاً في المتغير الاول كان اول ثلاثة حروف تدل علي ان نوعية البيانات من النوع التاريخ والمتغير الثاني كان او ثلاثة حروف في اسم المتغير تدل علي ان نوع البيانات التي ستدخل الي المتغير من النوع رقم integer

ومن الممكن بعد ذلك ان نقوم بإعطاء قيمة للمتغير او ان نقوم بتفريغ المتغير من القيمة التي بداخله عن طريق اعطاءه القيمة nothing ويظهر هذا في الشكل التالي فبعد تفريغ المتغير من القيمة التي كانت بداخله لا يظهر شيء اثناء الطباعة



٥- كيفية الاعلان عن Enumerations

Enumerations

احياناً اثناء التعامل بداخل الكود قد نود ان نقوم بتخزين مجموعة من القيم معاً في داخل مخزن واحد بحيث نستطيع استدعاء هذه القيم عن طريق اسماءها او عن طريق الرقم الخاص بها ، فمثلاً في مثال تغير لون خلفية سطر الاوامر نلاحظ انه عند الاختيار من بين الالوان الموجودة تظهر لنا قائمة بها مجموعات الالوان المختلفة وهذه القائمة هي ما يطلق عليه Enumerations وقد نود ان نقوم بعمل مثلها في البرنامج الخاص بنا

```
Module Module1

    Sub Main()

        Console.BackgroundColor = ConsoleColor.Blue
        Console.Clear()

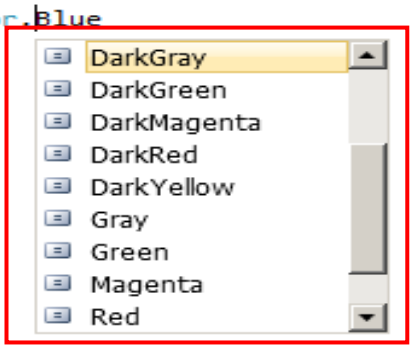


        Console.ReadKey()



    End Sub

End Module
```



فمن الممكن ايضاً ان نقوم باستدعاء احد القيم عن طريق الرقم الخاص به كما في المثال التالي:

```
Module Module1

    Sub Main()

        Console.BackgroundColor = 9
        Console.Clear()



        Console.ReadKey()



    End Sub

End Module
```

وطريقة انشاء هذه المجموعة من القيم عن طريق الامر Enum ثم نقوم بوضع القيم بداخله كلاً علي حسب القيمة الرقمية الخاص به والاسم الذى نود استدعاء تلك القيمة عن طريقه كما في الصورة التالية:

```
Module1                                                    Main
Module Module1

    Enum MyColor As Integer

        W = 15
        B = 1
        R = 12

    End Enum


    Sub Main()


        MyColor.
        Console.                    ndColor = ConsoleColor.RedBlue
        Console.
                    B
                    R
                    W

        Console.ReadKey()
```

فمثلاً هنا قمنا باستدعاء اللون عن طريق دلالة الاسم الخاص به في القائمة

```
Module1                                                    Main
Module Module1

    Enum MyColor As Integer

        W = 15
        B = 1
        R = 12

    End Enum


    Sub Main()


        Console.BackgroundColor = MyColor.R
        Console.Clear()


        Console.ReadKey()
```

وهنا قمنا بعمل قائمة بها قيمتين الاولي male والثانية female تحمل الاولي القيمة العددية ١ والثانية القيمة العددية ٢ وعندما يتم استدعاء الخاص باي واحدة منها فانه يقوم باستبدال الاسم بالرقم المصاحب له

```
Module Module1

    Enum ty As Integer

        Male = 1
        Fmale = 2

    End Enum


    Sub Main()



        ty.Fmale

        Console.Clear()


        Console.ReadKey()
```

وفي هذا المثال تظهر القيم الموجودة بداخل القائمة عندما يتم استدعاءها

```
Module1
Module Module1

    Enum WN As Integer

        Sat = 1
        Sun = 2


        Fri = 7
    End Enum


    Sub Main()

WN.
        ole.Clear()
        Fri
        Sat
        Sun
        Console.ReadKey()
```

وهنا من الممكن طباعة القيمة الموجودة بها عن طريق استدعاء الاسم المصاحب لها وارسال القيمة الي سطر الاوامر لطباعتها علي الشاشة.

```
Module Module1

    Enum WN As Integer

        Sat = 1
        Sun = 2


        Fri = 7
    End Enum


    Sub Main()


        Console.Write(WN.Fri)

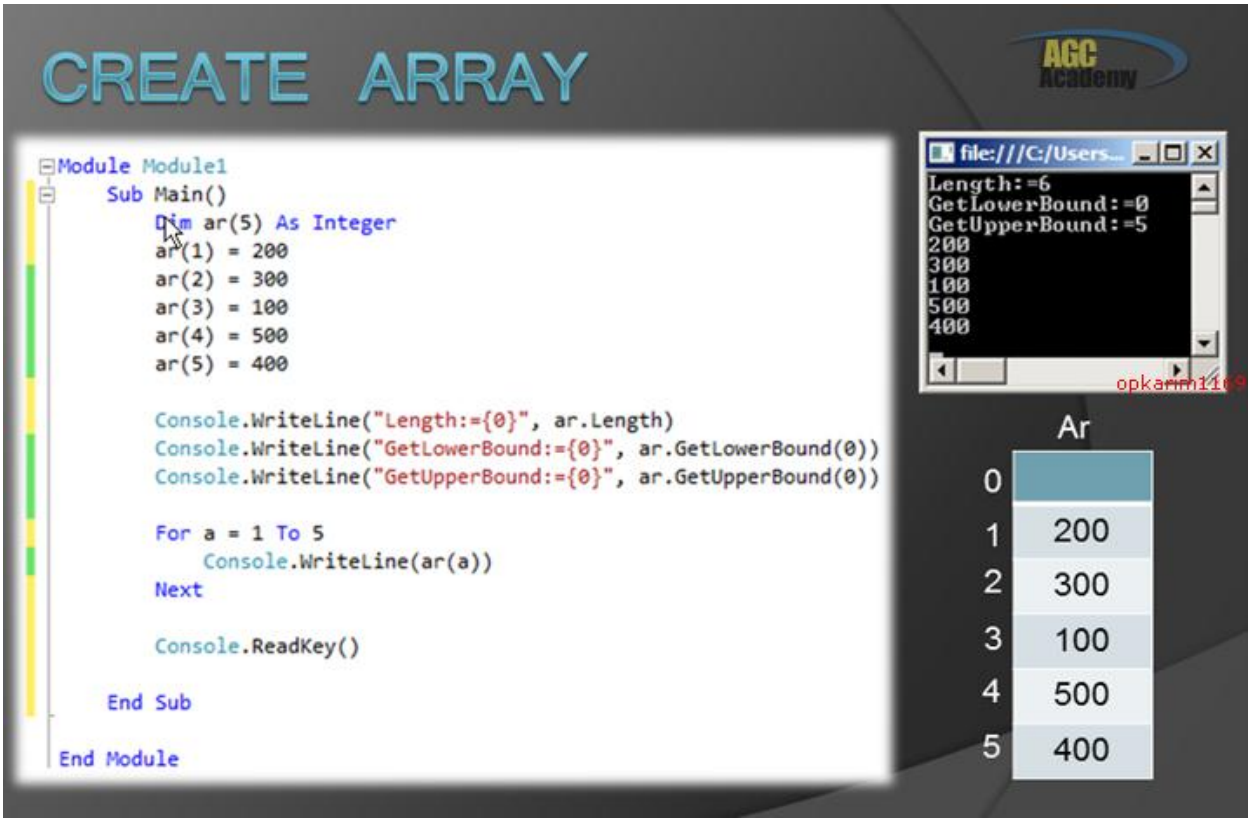
        Console.ReadKey()
```

٦- التعامل مع المصفوفات



المصفوفات هي عبارة عن مجموعة من المتغيرات تكون محصورة بداخل قائمة لكل متغير فيها رقم index وتكون هذه المتغيرات تحمل نوعية واحدة من البيانات فمثلاً في المثال القادم تكون المصفوفة من النوع integer اي ان جميع المتغيرات بداخلها ستكون من النوع الرقمي.

وعملية انشاء المصفوفة تتم مثل انشاء المتغير العادي عن طريق الامر dim ولكن تختلف فقط في انها يتم وضع قوسين بعد اسم المصفوفة يوجد بداخلهم عدد عناصر المصفوفة وبعد ذلك من الممكن ان نقوم بعمل تخصيص للقيم بداخل المصفوفة او ان يتم الامر بعد انشاءها وفي المثال في الصورة التالية قمنا بإنشاء مصفوفة اسمها a وطولها ٥ من النوع الرقمي وبعد ذلك قمنا بطباعة طول المصفوفة وطول اصغر واكبر حدود المصفوفة ثم قمنا بطباعة جميع عناصر المصفوفة بدلالة رقم العنصر.



فمثلاً في الصورة القادمة قمنا بالإعلان عن مصفوفة طولها ٥ وقمنا بعد ذلك بطباعة القيمة الموجودة بداخل المتغير الثالث في المصفوفة وهو في هذه الحالة ٧٠٠

```
Module Module1

    Sub Main()

        Dim M(5) As Integer
        M(1) = 100
        M(2) = 500
        M(3) = 700
        M(4) = 50
        M(5) = 250

        Console.WriteLine(M(3))
        Console.ReadKey()

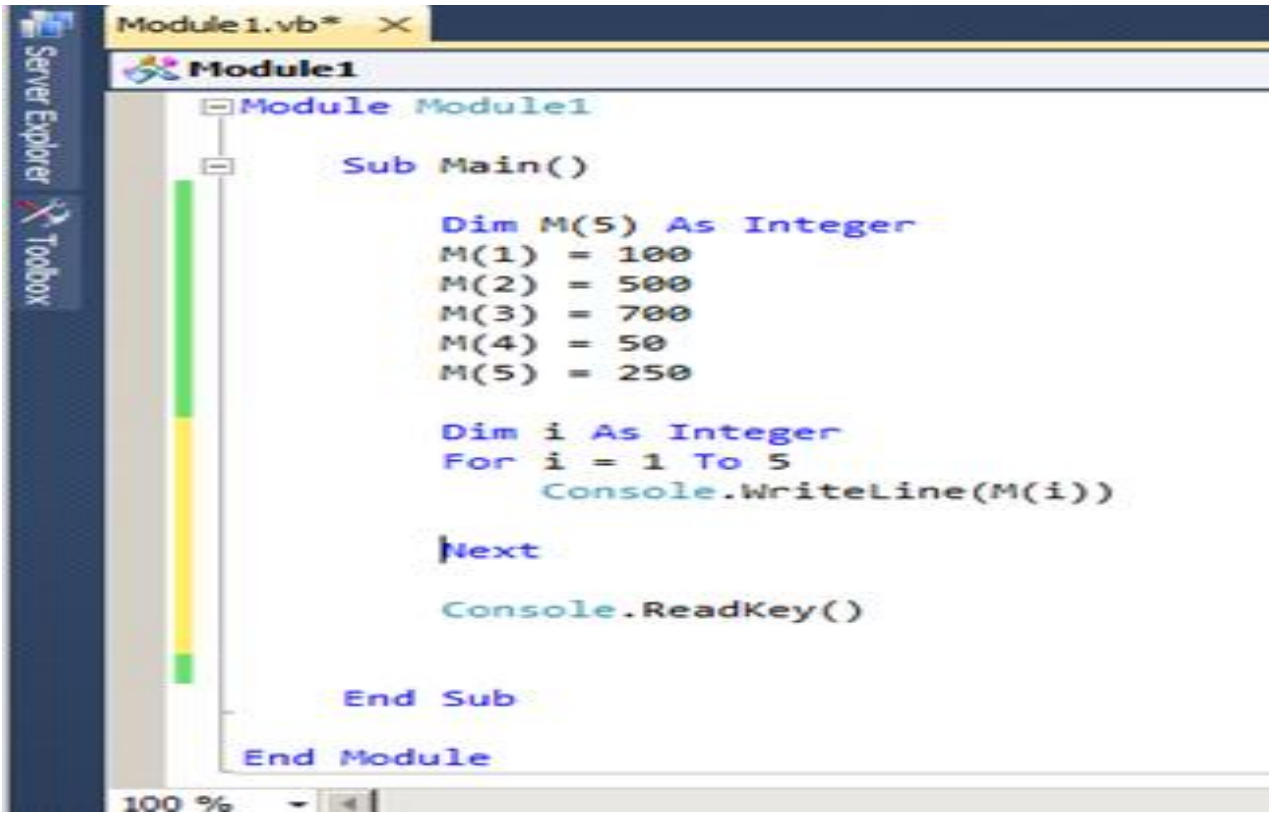

    End Sub

End Module
```

وتظهر القيمة في شاشة سطر الاوامر كما في الشكل التالي:



ومن اجل ان نقوم بطباعة جميع قيمة العناصر الموجودة بداخل المصفوفة لابد وان نقوم بالمرور علي جميع ارقام البحث الخاصة بالمتغيرات وقمنا بعمل ذلك عن طريق اللوب القادم.



٧- ادخال قيم في المصفوفة:

اولاً قمنا بإنشاء المصفوفة وقمنا بوضع خمس عناصر بداخلها.

```
Module Module1

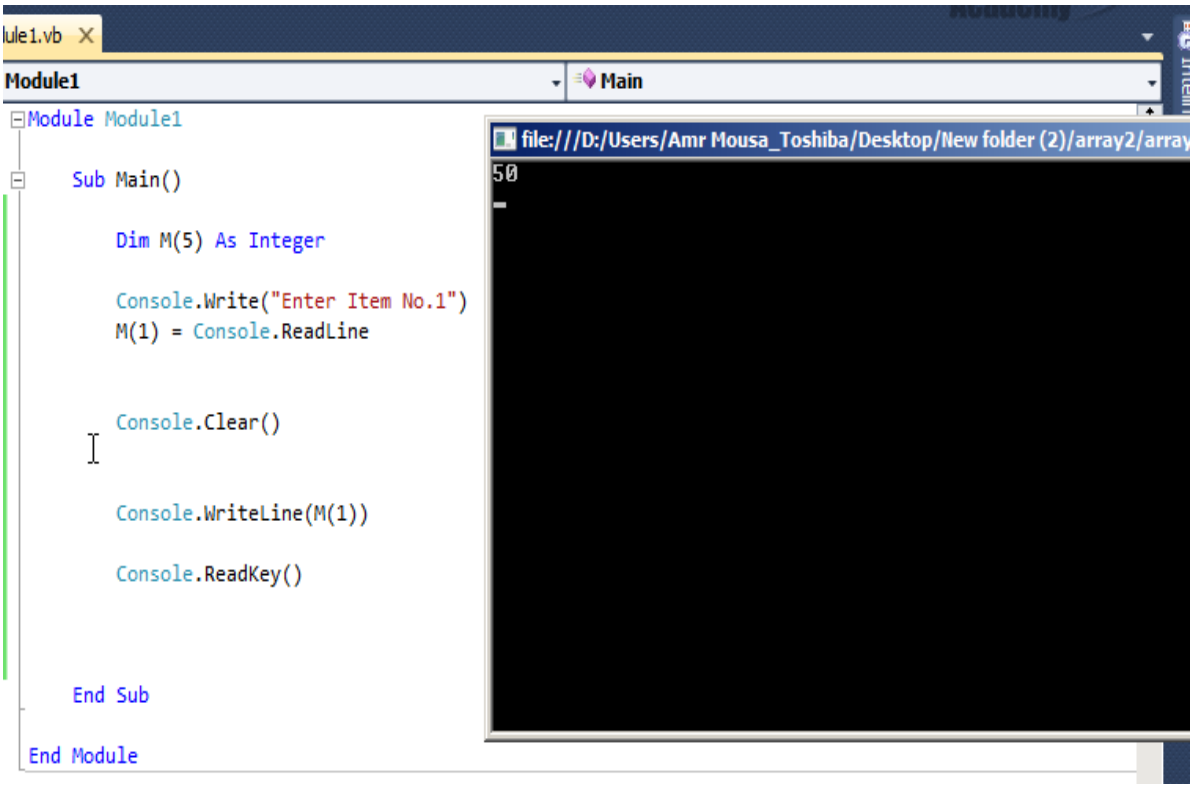
    Sub Main()

        Dim M(5) As Integer
        M(1) = 200
        M(2) = 700
        M(3) = 1000
        M(4) = 150
        M(5) = 600

    End Sub

End Module
```

نقوم بعد ذلك بسؤال المستخدم عن القيمة التي يود ادخالها في المتغير الاول في المصفوفة ثم نقوم بعمل مسح لشاشة سطر الاوامر من مجموعات الاوامر السابقة عن طريق الامر Console.Clear() ثم نقوم بكتابة القيمة الموجودة داخل المتغير الاول في المصفوفة التي قام المستخدم بإدخالها. ويظهر هذا في الشكل التالي:



وفي مجموعات الاكواد التالية سنقوم بسؤال المستخدم عن جميع القيم التي يود ادخالها في جميع المتغيرات الموجودة بداخل المصفوفة ويتم بعد ذلك عرض تلك القيم التي قام المستخدم بإدخالها عن طريق سطر الاوامر.

```
Sub Main()

    Dim M(5) As Integer

    Dim i As Integer

    For i = 1 To 5
        Console.Write("Enter Item No {0} ", i)
        M(i) = Console.ReadLine
    Next

    Console.Clear()

    For i = 1 To 5
        Console.WriteLine(M(i))
    Next


    Console.ReadKey()
```

٩- امر الترتيب والعكس sort and reverse

بداية الكود السابق يقوم بسؤال المستخدم عن القيم التي يقوم بإدخالها في المصفوفة وسنقوم بتوضيح الاوامر اكثر عن طريق كتابة بعض التعليقات عليها:

```
        Dim M() As Integer
        ReDim M(t)

        Dim i As Integer

        For i = 1 To t
            Console.Write("Enter Item No {0} ", i)
            M(i) = Console.ReadLine
        Next

        Console.Clear()

        For i = 1 To t
            Console.WriteLine(M(i))
        Next

        Console.ReadKey()

    End Sub

End Module
```

Main

```
Console.Write("Enter Total Noumbers:")
t = Console.ReadLine

'مسح الشاشة
Console.Clear()

' انشاء المصفوفة
Dim M() As Integer
ReDim M(t)

' كود سؤال المستخدم عن الارقام
Dim i As Integer
For i = 1 To t
    Console.Write("Enter Item No {0} ", i)
    M(i) = Console.ReadLine
Next

' طباعة بيانات المصفوفة مره اخرى على الشاشة
For i = 1 To t
    Console.WriteLine(M(i))
Next

Console.ReadKey()
```

```
        Console.Clear()


        '-------------------------------------------------------------
        ' انشاء المصفوفة
        Dim M() As Integer
        ReDim M(t)
        '
        ' كود سؤال المستخدم عن الارقام
        Dim i As Integer
        For i = 1 To t
            Console.Write("Enter Item No {0} ", i)
            M(i) = Console.ReadLine
        Next

        ' طباعة بيانات المصفوفة مره اخرى على الشاشة
        For i = 1 To t
            Console.WriteLine(M(i))
        Next

        Console.ReadKey()

    End Sub

    '-------------------------------------------------------------
    ' كود سؤال المستخدم عن الارقام
    Dim i As Integer
    For i = 1 To t
        Console.Write("Enter Item No {0} ", i)
        M(i) = Console.ReadLine
    Next
    '-------------------------------------------------------------


    '-------------------------------------------------------------
    ' طباعة بيانات المصفوفة مره اخرى على الشاشة
    For i = 1 To t
        Console.WriteLine(M(i))
    Next
    '-------------------------------------------------------------


    Console.ReadKey()
```

وهنا نقوم بعد ادخال المستخدم للقيم التي ستحفظ في داخل المصفوفة نقوم بعمل ترتيب لهذه القيم عن طريق استخدام الامر

System.Array.Sort(M) حيث M هي المصفوفة التي نود ترتيب عناصرها

```
Dim M() As Integer
ReDim M(t)
'-----------------------------------------------------------------



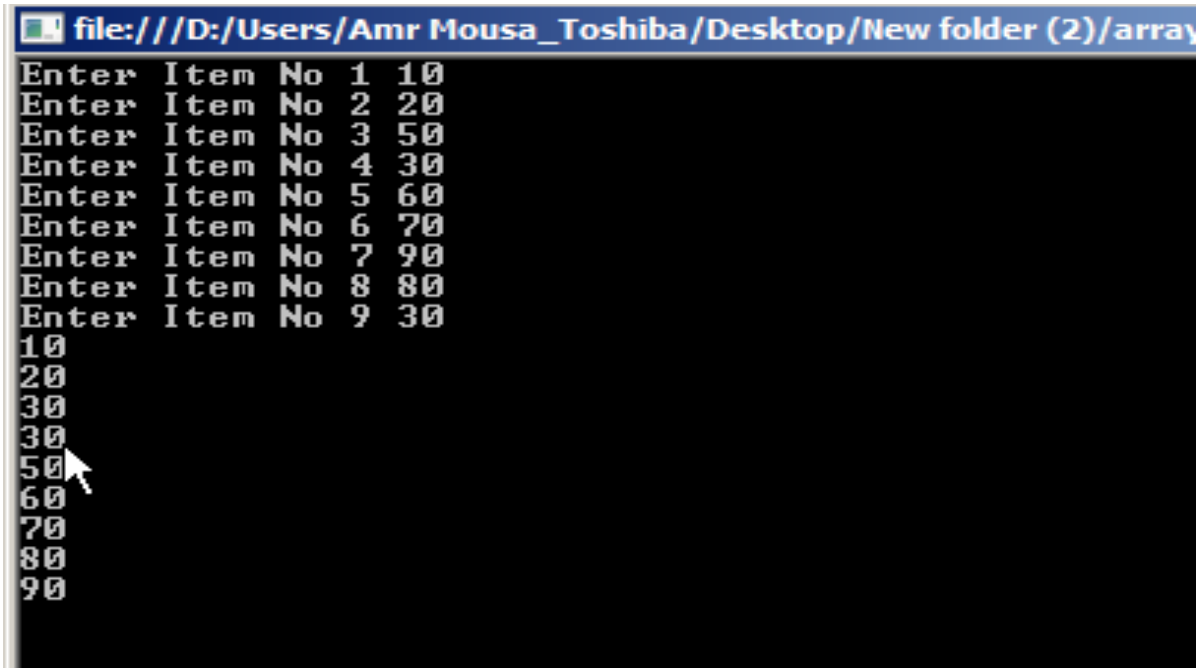
'-----------------------------------------------------------------
' كود سؤال المستخدم عن الارقام
Dim i As Integer
For i = 1 To t
    Console.Write("Enter Item No {0} ", i)
    M(i) = Console.ReadLine
Next
'-----------------------------------------------------------------

System.Array.Sort(M)



'-----------------------------------------------------------------
' طباعة بيانات المصفوفة مره اخرى على الشاشة
For i = 1 To t
    Console.WriteLine(M(i))
```

وعند تنفيذ البرنامج نلاحظ انه بعد ادخال البيانات تم طباعتها وقد تم ترتيبها تصاعدياً



الامر التالي هو امر العكس وهو بعد ان نقوم بعمل ترتيب للقيم بداخل المصفوفة قد نود ان نقوم بعكس تلك القيم بحيث يكون الاصغر فيها هو اخر متغير في المصفوفة فيتم هذا عن طريق الامر System.Array.Reverse(M) كما في الصورة التالية:

```
e1                                                    Main
    For i = 1 To t
        Console.Write("Enter Item No {0} ", i)
        M(i) = Console.ReadLine
    Next
    '-----------------------------------------------------------------

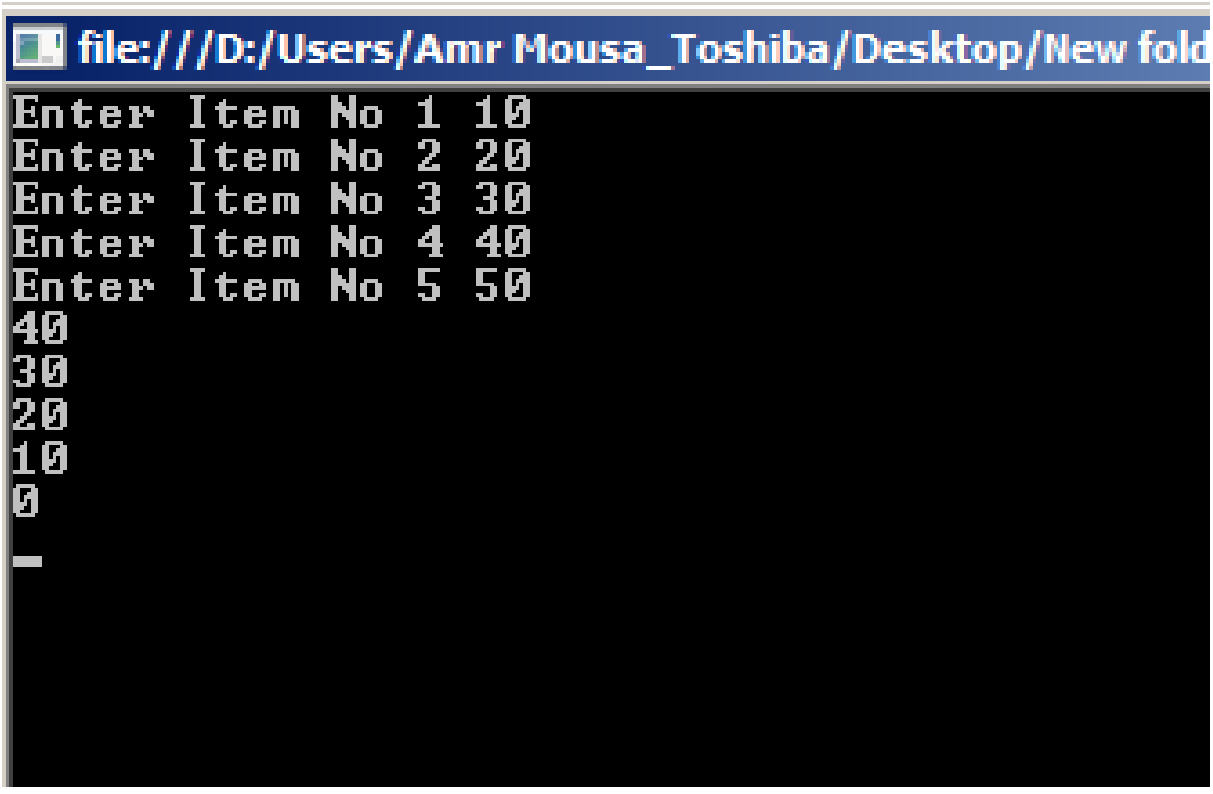
    System.Array.Sort(M)

    System.Array.Reverse(M)



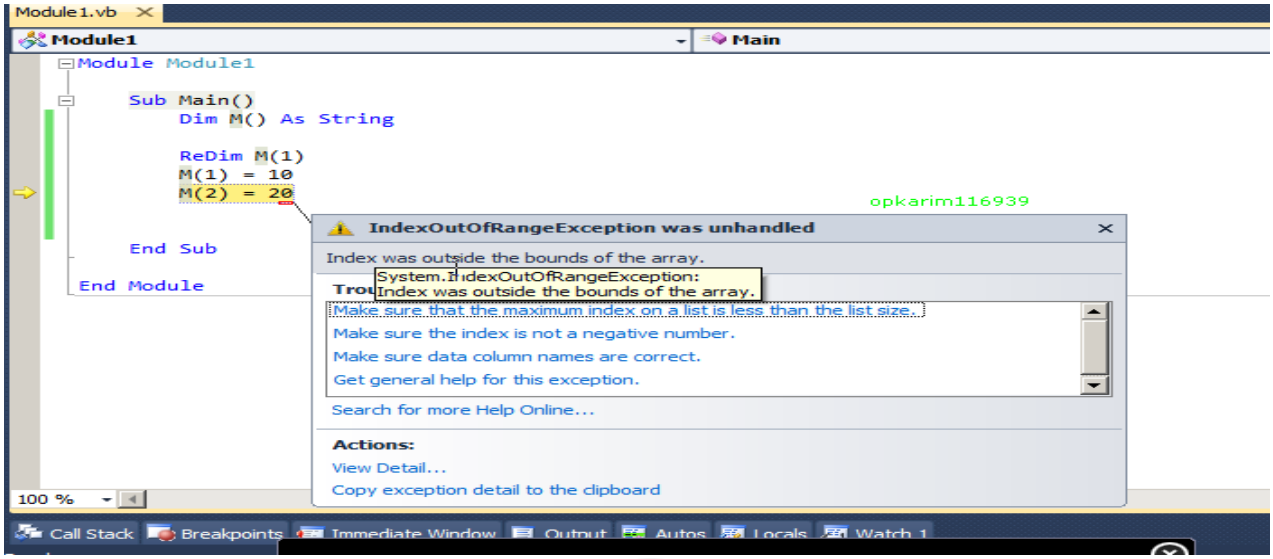
    '-----------------------------------------------------------------
    ' طباعة بيانات المصفوفة مره اخرى على الشاشة
    For i = 1 To t
        Console.WriteLine(M(i))
    Next
    '-----------------------------------------------------------------
```

وتظهر البيانات وقد تم طباعتها بشكل عكسي



١٠- الفرق ما بين عدد عناصر المصفوفة وطولها وحذف المصفوفة:

اولاً لابد وان نوضح ان طول المصفوفة يختلف عن عدد العناصر بها فعندما نقوم بالإعلان عن مصفوفة ونقوم بوضع الرقم داخل القوسين بعد اسم المصفوفة فإن هذا الرقم يحدد عدد العناصر التي تكون في المصفوفة ولكن طول المصفوفة هو عدد المخازن التي يمكن تخزين البيانات بداخلها في داخل المصفوفة وطول المصفوفة دائماً يكون اكبر من عدد عناصر المصفوفة بواحد ومن الممكن التعرف عن طول المصفوفة عن طريق الامر length و اماكن تخزين المتغيرات بداخل المصفوفة تبدأ بالرقم صفر وتنتهي عند العدد الذى قمنا بإدخاله عن الاعلان عن المصفوفة فمثلاً في المثال القادم قمنا بالإعلان عن مصفوفة من النوع النصي ولم نقم بإدخال عدد لها وبعد الاعلان عنها قمنا باستخدام الامر ReDim من اجل اعادة ادخال عدد العناصر في المصفوفة وقمنا بوضع العدد ١ وهذا يعني ان طول المصفوفة الصحيح هو ٢ ولكن عندمنا قمنا بمحاولة اضافة قيمة في المخزن رقم ٢ في المصفوفة قام البرنامج بإعطائنا خطأ بأن هذا الرقم غير موجود والسبب في ذلك نعم ان عدد العناصر الموجودة بداخل المصفوفة هو اثنين ولكن عندما نقوم بالدلالة اليها او بالإشارة اليه من اجل تخزين قيمة بداخل احدهم لابد وان نبدأ العدد من صفر وفي هذه الحالة يتم الاشارة الي العناصر في داخل المصفوفة عن طريق الرقم صفر و واحد وليس ٢ فاكبر رقم للبحث او الفهرسة بداخل المصفوفة هو ١ علي الرغم من ان طولها هو ٢ بسبب اننا نبدأ العد من رقم صفر وليس من واحد



ملاحظة ان الامر ReDim يقوم بإنشاء المصفوفة من اول وجديد وبالتالي فإنه في تلك الحالة يتم حذف القيم التي تكون بداخل المتغيرات في المصفوفة فمثلاً في المثال التالي نلاحظ انه قد تم انشاء المصفوفة وادخال قيم في المتغير الاول والثاني ولكن بعد تغير حجم المصفوفة بالزيادة الي خمسة وادخال قيم اخري في المتغيرات ٣ و٤ و٥ هذا نتج عنه ان المتغيرات الاول والثاني قد تم حذف القيم من داخلهم وهذا يظهر في حالة طباعة جميع العناصر الموجودة بداخل المصفوفة انه قد تم طباعة فقط العناصر من الثالث وحتى الاخير وحذف القيم الأخرى وهذا بسبب ان تغير حجم المصفوفة عن طريق الامر ReDim ينتج عنه حذف القيم السابقة فيها.

```
Sub Main()
    Dim M() As String

    ReDim M(2)
    M(1) = 10
    M(2) = 20

    ReDim M(5)
    M(3) = 100
    M(4) = 200
    M(5) = 300

    Dim i As Integer = 0
    For i = 1 To 5
        Console.WriteLine(M(i))

    Next

    Console.ReadKey()

End Sub
```



وفي حالة الرغبة في الاحتفاظ بالعناصر السابقة يتم اضافة الامر Preserve

```
Sub Main()
    Dim M() As String

    ReDim M(2)
    M(1) = 10
    M(2) = 20

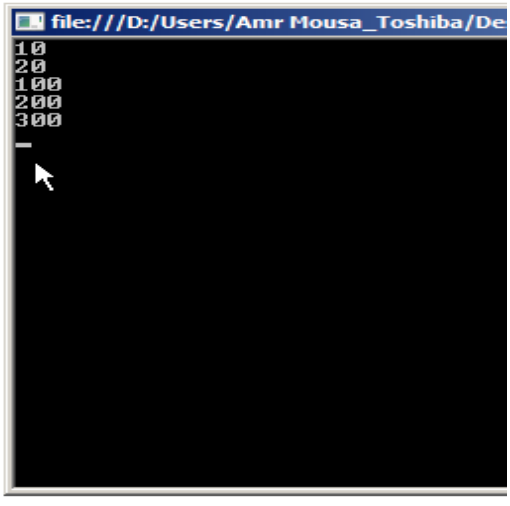
    ReDim Preserve M(5)
    M(3) = 100
    M(4) = 200
    M(5) = 300

    Dim i As Integer = 0
    For i = 1 To 5
        Console.WriteLine(M(i))

    Next

    Console.ReadKey()

End Sub
```



وعند استخدام الامر Preserve يفضل الاحتفاظ بالطول السابق للمصفوفة واضافة الطول الجديد اليها.

```
Sub Main()
    Dim M() As String

    ReDim M(2)
    M(1) = 10
    M(2) = 20

    ReDim Preserve M(M.Length +1)
    M(3) = 100
    M(4) = 200
    M(5) = 300

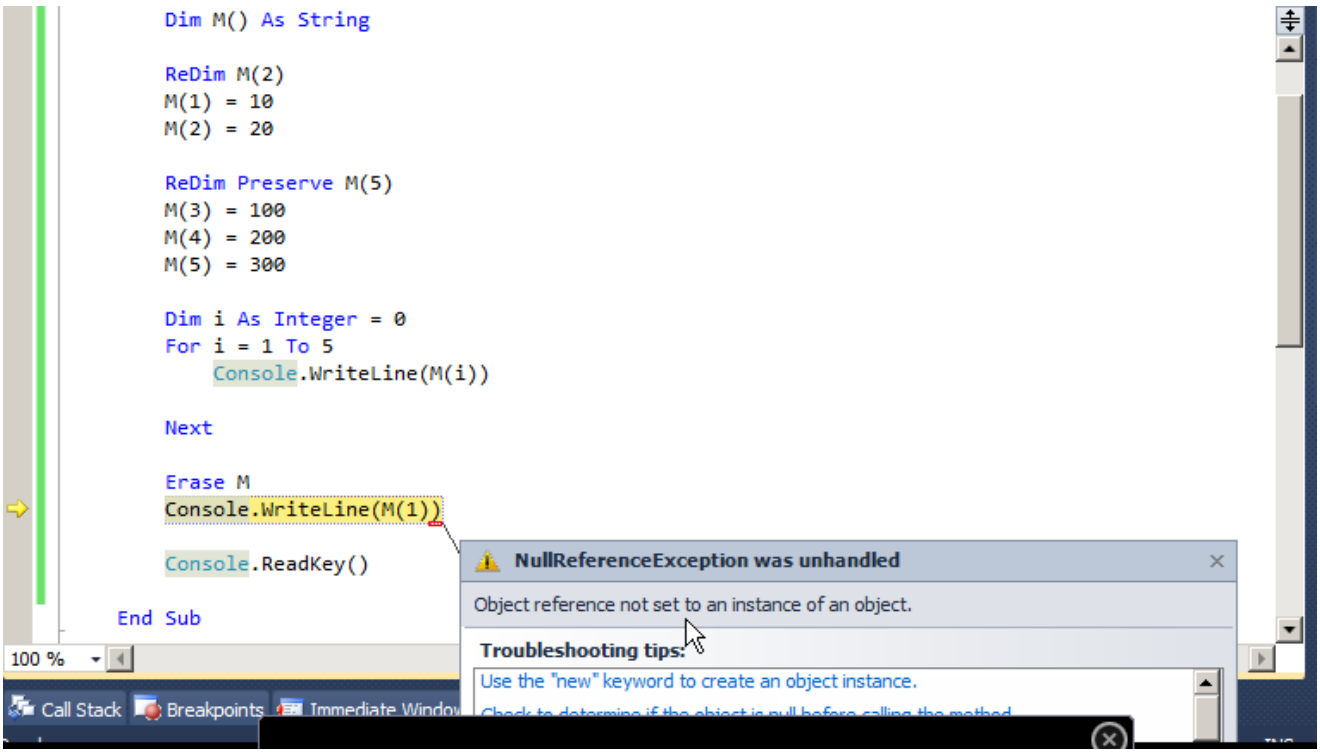
    Dim i As Integer = 0
    For i = 1 To 5
        Console.WriteLine(M(i))

    Next

    Console.ReadKey()

End Sub
```

من اجل حذف المصفوفة نقوم باستخدام الامر Erase وفي حالة حذف المصفوفة لا يمكن الاشارة اليها بعد ذلك.



٣١- المعاملات الاختيارية Optional Parameter

يقصد بالمعاملات الاختيارية هي المعاملات التي تكون في تعريف الدالة ويجوز عند استدعاء الدالة الا اقوم بإرسال قيمة لها وفي تلك الحالة تقوم الدالة بوضع قيمة افتراضية في تلك المعاملات.

فمثلاً في المثال التالي قمنا بوضع مفتاح اضافي الي دالتي التشفير وفك التشفير وهذا المفتاح اسمه هو Key عند استدعاء الدالة لابد وان نرسل قيمة هذا المفتاح اليها كي تستخدمه في عملية تشفير وفك تشفير النصوص.

```
Module1                                                    Main
Module Module1
    Sub Main()

        Dim K As String = "I Love AGC Academy"

        Dim En As String = MyEnc(K, "!@#")
        Dim De As String = MyDecryption(En, "!@#")

        Console.WriteLine(En)
        Console.WriteLine(De)

        Console.ReadKey()

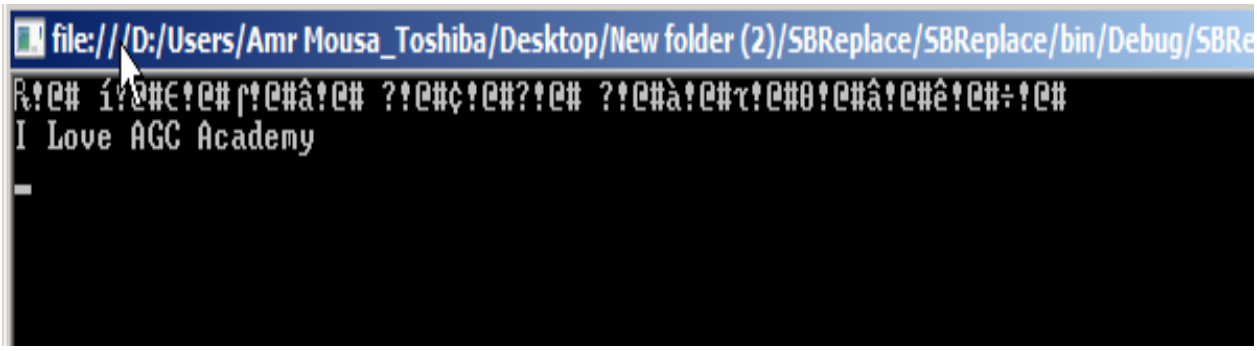
    End Sub

    Function MyEnc(ByVal msg As String, ByVal key As String) As String ...

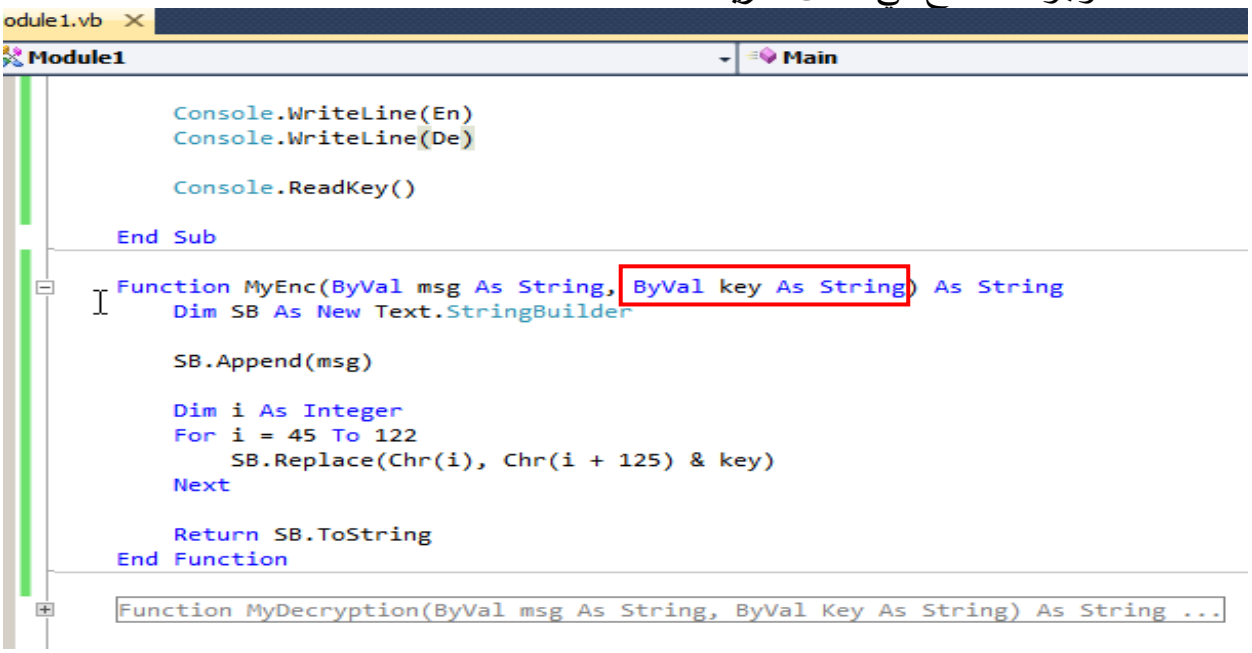
    Function MyDecryption(ByVal msg As String, ByVal Key As String) As String ...

End Module
```

نلاحظ هنا تغير شكل النصوص بإضافة المفتاح في نهاية كل حرف



نلاحظ وجود المفتاح في داخل تعريف الدالة



ما سنقوم بفعله بعد ذلك هو وضع قيمة افتراضية لهذا المتغير عن طريق اضافة كلمة Optional الي تعريف المتغير وفي النهاية اضافة القيمة التي نود ان تكون موجودة في حالة عدم ارسال قيمة له اثناء استدعاء الدالة.

```
Function MyEnc(ByVal msg As String, Optional ByVal key As String = "!@#") As String
    Dim SB As New Text.StringBuilder

    SB.Append(msg)

    Dim i As Integer
    For i = 45 To 122
        SB.Replace(Chr(i), Chr(i + 125) & key)
    Next

    Return SB.ToString
End Function
```

```
Function MyDecryption(ByVal msg As String, ByVal Key As String) As String ...
```

بعد ذلك نقوم باستدعاء دالة التشفير دون ارسال قيمة للمتغير key ثم نقوم بإرسال نفس القيمة الافتراضية لدالة فك التشفير كما في الشكل التالي:

```
Module1                                                    Main
    Sub Main()

        Dim K As String = "I Love AGC Academy"

        Dim En As String = MyEnc(K)
        Dim De As String = MyDecryption(En, "!@#")

        Console.WriteLine(En)
        Console.WriteLine(De)

        Console.ReadKey()

    End Sub

    Function MyEnc(ByVal msg As String, Optional ByVal key As String = "!@#") As String ...

    Function MyDecryption(ByVal msg As String, ByVal Key As String) As String ...

End Module
```

ومن الممكن ايضاً اضافة نفس المتغير الاختياري الي دالة فك التشفير ففي تلك الحالة من الممكن الا احتاج الي ارسال قيمة المتغيرات الي الدوال فتأخذ القيمة الافتراضية الموجودة بداخلها.

```
        Dim K As String = "I Love AGC Academy"

        Dim En As String = MyEnc(K)
        Dim De As String = MyDecryption(En)

        Console.WriteLine(En)
        Console.WriteLine(De)

        Console.ReadKey()

    End Sub

    Function MyEnc(ByVal msg As String, Optional ByVal key As String = "#$%") As String ...

    Function MyDecryption(ByVal msg As String, Optional ByVal Key As String = "#$%") As String ...

End Module
```

وفي حالة الرغبة في تعديل القيم التي بداخل المتغيرات يتم هذا الامر بسهولة عن طريق ارسال القيم اليها اثناء الاستدعاء مثل الطريقة القديمة في تلك الحالة يتم تجاهل القيمة الافتراضية الموجودة بداخل الدالة ويتم تعيين القيمة الجديدة الي تلك المتغيرات.

```
        Dim K As String = "I Love AGC Academy"

        Dim En As String = MyEnc(K, "%^&")

        Dim De As String = MyDecryption(En, "%^&")

        Console.WriteLine(En)
        Console.WriteLine(De)

        Console.ReadKey()

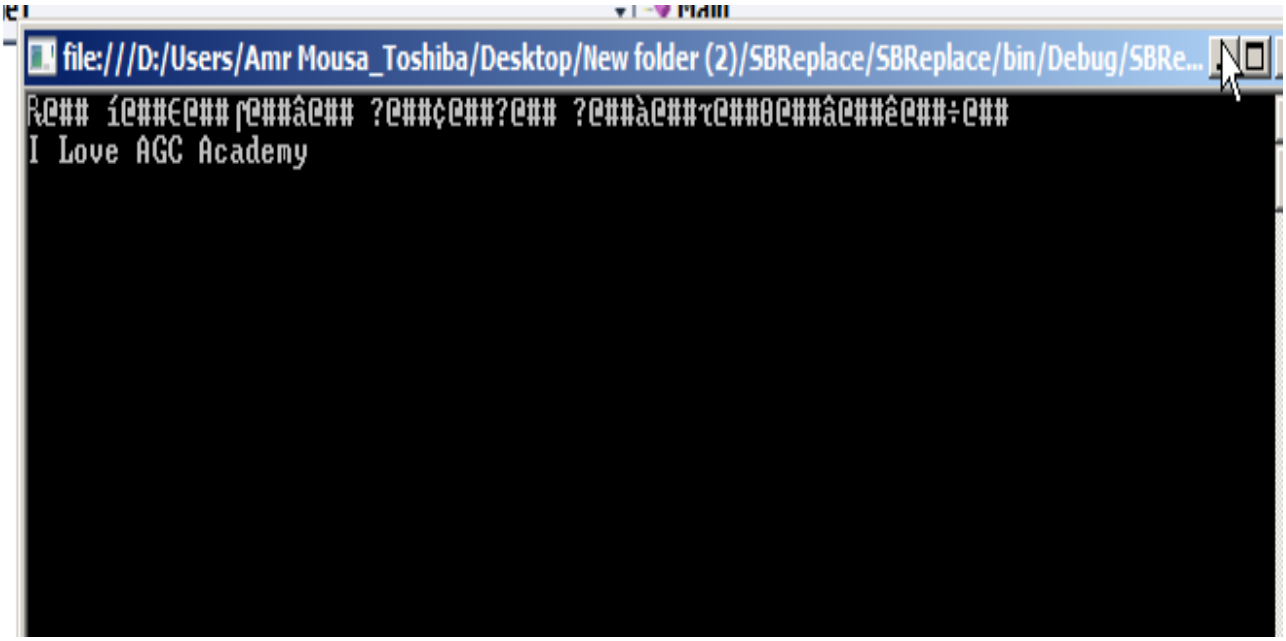
    End Sub

    Function MyEnc(ByVal msg As String, Optional ByVal key As String = "#$%") As String ...

    Function MyDecryption(ByVal msg As String, Optional ByVal Key As String = "#$%") As String ...

End Module
```

ويتم تشفير وفك تشفير النصوص بشكل طبيعي عن طريق المفاتيح التي قمنا بإرسالها الي الدالتين ولابد من ان تكون قيمة المفتاحين متساوية في الدالتين



٣٢- Functions Overloading

يقصد بهذا الموضوع انه من الممكن ان يكون هناك اكثر من دالة تحمل نفس الاسم وتختلف فقط في انواع المعاملات التي تأخذها فمثلاً في الصورة التالية هناك دالتين يستخدمان في تشفير البيانات والاثنين يحملان نفس الاسم ولكن الاختلاف بينهما في البنية الداخلية للدالة وفي عدد المعاملات التي تأخذها كل دالة من هذه الدوال ففي الدالة الاولي نلاحظ ان عدد المعاملات واحد والثانية عدد المعاملات فيها اثنين وفي تلك الحالة يتم معرفة الدالة المطلوب استدعاءها عن طريق عدد المعطيات التي تأخذ الدالة اثناء استدعاءها

```
Function MyEnc(ByVal msg As String) As String
    Dim SB As New Text.StringBuilder

    SB.Append(msg)

    Dim i As Integer
    For i = 45 To 122
        SB.Replace(Chr(i), Chr(i + 125) & "@#$")
    Next

    Return SB.ToString
End Function

Function MyEnc(ByVal msg As String, ByVal key As String) As String
    Dim SB As New Text.StringBuilder

    SB.Append(msg)

    Dim i As Integer
    For i = 45 To 122
        SB.Replace(Chr(i), Chr(i + 125) & key)
    Next
```

فمثلاً هنا قمنا باستدعاء الدالة الاولي التي تحمل معامل واحد

```
        Dim K As String = "I Love AGC Academy"

        Dim En As String = MyEnc(K)

        Dim De As String = MyDecryption(En, "@##")

        Console.WriteLine(En)
        Console.WriteLine(De)

        Console.ReadKey()

    End Sub

    Function MyEnc(ByVal msg As String) As String ...
    Function MyEnc(ByVal msg As String, ByVal key As String) As String ...

    Function MyDecryption(ByVal msg As String) As String ...
    Function MyDecryption(ByVal msg As String, ByVal Key As String) As String ...
End Module
```

وفي حالة الرغبة في استدعاء الدالة الثانية يتم ذلك عن طريق ارسال معاملين اثنين الي نفس اسم الدالة.

وفي حالة اننا قمنا بالخطأ بإرسال عدد اكبر من المعاملات ستظهر رسالة خطأ تظهر لنا عدم امكانية التعرف علي الدالة المطلوبة.

```
Module Module1
    Sub Main()

        Dim K As String = "I Love AGC Academy"

        Dim En As String = MyEnc(K, "#$%", "eee")

        Dim De As String = MyDecryption(En, "#$%

        Console.WriteLine(En)
        Console.WriteLine(De)

        Console.ReadKey()

    End Sub

    Function MyEnc(ByVal msg As String) As String ...
    Function MyEnc(ByVal msg As String, ByVal key As String) As String ...

    Function MyDecryption(ByVal msg As String) As String ...
    Function MyDecryption(ByVal msg As String, ByVal Key As String) As String ...
End Module
```

Overload resolution failed because no accessible 'MyEnc' accepts this number of arguments.

Generate method stub for 'MyEnc' in 'SBReplace.Module1'

Generate property stub for 'MyEnc' in 'SBReplace.Module1'

وهناك ملحوظة هامة حول هذا الامر ظهرت لي اثناء التجربة وهي ان الفيجوال بيزك لا يقبل التفرقة ما بين الدوال عن طريق فقط المعطيات الاختيارية Optional Parameters اذا تساوت عدد المعطيات فمثلاً في المثال السابق اذا كان المعطي Key في الدالة MyEnc من النوع الاختياري Optional وكان هذا هو المعطي الوحيد المختلف في التفرقة ما بين الدالتين فلن تقبل الفيجوال بيزك هذه العملية وستظهر رسالة خطأ تفيد عدم امكانية اتمام عملية الـ Overloading .

ومن الممكن اضافة اي عدد نريده من الدوال بنفس الاسم مادام سيتم التفرقة بينهم عن طريق عدد المعطيات في كل منهم ففي هذا المثال لدينا ثلاث دوال بنفس الاسم وقمنا فقط باستدعاء الدالة التي نريد التعامل معها عن طريق عدد المعاملات.

```
Module Module1
    Sub Main()

        Console.WriteLine(Avg(10, 15))
        Console.ReadKey()

    End Sub

    Function Avg(ByVal X1 As Double, ByVal X2 As Double, ByVal X3 As Double) As Double ...

    Function Avg(ByVal X1 As Double, ByVal X2 As Double) As Double ...

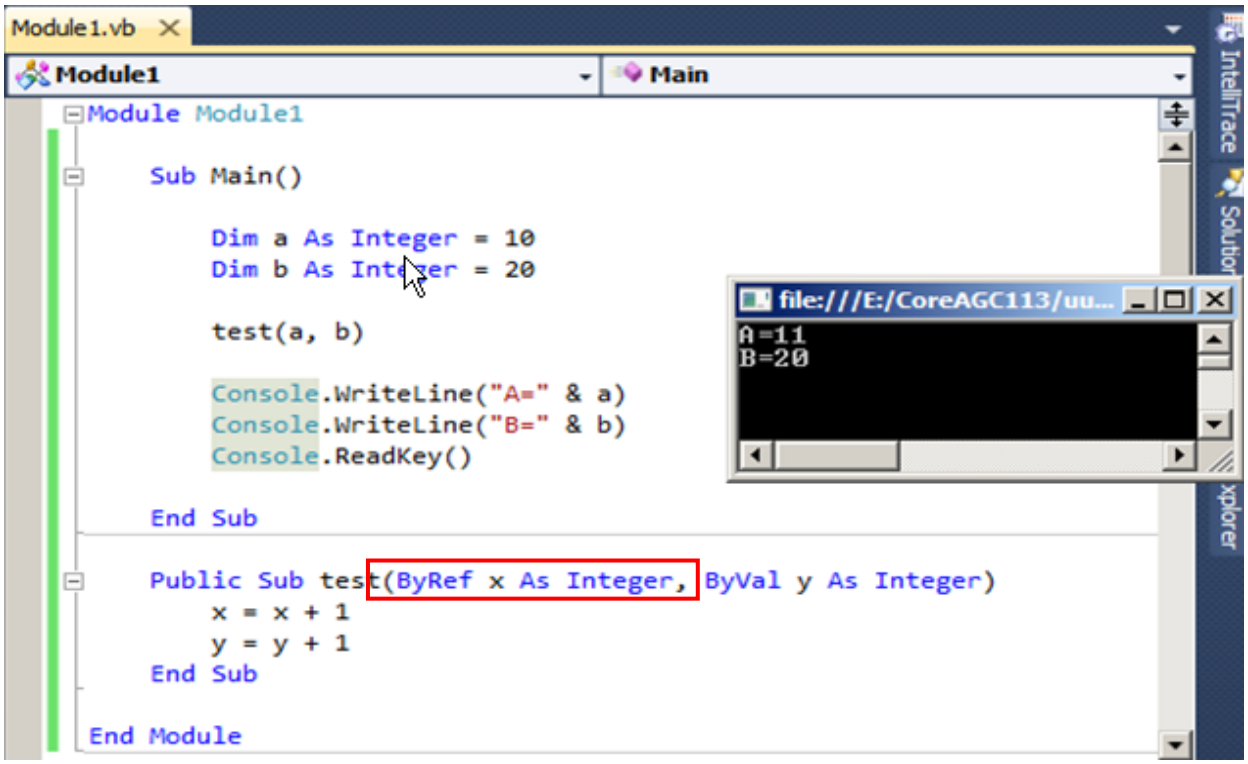
    Function Avg(ByVal X1 As Double) As Double ...
End Module
```

٣٤- الفرق ما بين ByVal and ByRef



عندما يتم انشاء الدالة يتم الاعلان عن المعاملات الخاصة بها اما عن طريق الاختيار ByVal أو ByRef والفرق ما بين الاثنين في انه عندما يتم ارسال قيمة الي المعامل الذى يأخذ الاختيار ByVal فهذا يعني انه يتم اخذ نسخة من قيمة المتغير والتعامل معها بداخل الدالة وعندها لا يتم التعديل علي القيمة الاصلية التي كان يحملها المتغير وتظل كما هي وعندما يكون المعامل يأخذ الاختيار ByRef يتم التعامل مع هذا المتغير نفسه وعندما يتم التعديل علي القيمة التي بداخله فإن القيمة الجديدة هي التي يظل المتغير محتفظ بها فـ ByVal تتعامل بأخذ نسخة من قيمة المتغير بينما ByRef تتعامل مع المؤشر او Reference الخاص بالمتغير فعندما يتم حدوث تغير يحدث هذا التغير في المتغير الاصلي وليس في نسخة منه.

ففي المثال القادم قمنا بإنشاء الدالة test وهي تستقبل معاملين اثنين وتقوم بالتعديل علي قيمتهم ولكن المعامل الاول يقوم بالتعديل علي اصل القيمة التي ارسلت له بينما الثاني يقوم بالتعديل علي نسخة من القيمة التي ارسلت له فعندما قمنا بإرسال المتغيرات a , b وقمنا بطباعة القيمة التي بداخلهم بعد عودة الدالة test نلاحظ ان قيمة المتغير الاول a هي التي تغيرات بينما ظلت قيمة المتغير الثاني كما هي والسبب في ذلك ان المتغير الاول قد تم الاشارة اليه عن طريق ByRef بداخل الدالة test فقامت الدالة بالتعديل علي القيمة الاصلية له.


Module1.vb
Module1
Main
Module Module1
Sub Main()
Dim a As Integer = 10
Dim b As Integer = 20
test(a, b)
Console.WriteLine("A=" & a)
Console.WriteLine("B=" & b)
Console.ReadKey()
End Sub
Public Sub test(ByRef x As Integer, ByVal y As Integer)
x = x + 1
y = y + 1
End Sub
End Module
file:///E:/CoreAGC113/uu...
A=11
B=20

٣٦- تمرير مصفوفة الي دالة:



سنقوم في هذا المثال بإنشاء دالة تأخذ مصفوفة كمعامل لها ثم تقوم بحساب المتوسط الخاص بعناصر هذه المصفوفة ونقوم اولاً بإنشاء المصفوفة ونقوم بحساب مجموع عناصر المصفوفة عن طريق الامر Sum

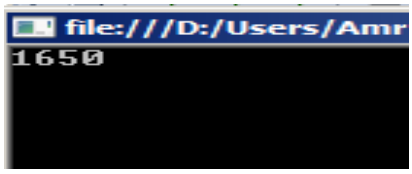
```
Module Module1
    Sub Main()

        Dim k(5) As Double
        k(1) = 100
        k(2) = 300
        k(3) = 250
        k(4) = 200
        k(5) = 800

        Dim D As Double

        D = k.Sum()
        Console.Write(D)
        Console.ReadKey()
```

طباعة مجموع عناصر المصفوفة



نقوم بعد ذلك بقسمة المجموع علي عدد عناصر المصفوفة

```
Module Module1
    Sub Main()

        Dim k(5) As Double
        k(1) = 100
        k(2) = 300
        k(3) = 250
        k(4) = 200
        k(5) = 800

        Dim D As Double

        D = k.Sum() / 5
        Console.Write(D)
        Console.ReadKey()
```

فتظهر القيمة في الصورة التالية:



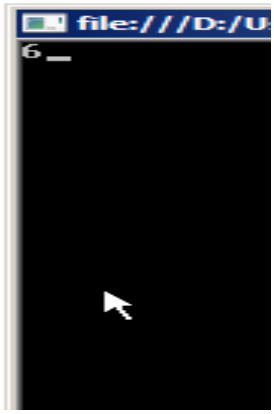
بعد ذلك نقوم باستخدام الامر Length بمعرفة طول المصفوفة او عدد العناصر الموجودة بداخل المصفوفة ولكن هناك ملاحظة هنا انه يتم اضافة العنصر الاول في المصفوفة الي طول المصفوفة او العنصر الذى يكون الاندكيس الخاص به يساوي صفر

```
Module Module1
    Sub Main()

        Dim k(5) As Double
        k(1) = 100
        k(2) = 300
        k(3) = 250
        k(4) = 200
        k(5) = 800

        Dim D As Double

        D = k.Length
        Console.Write(D)
        Console.ReadKey()
```



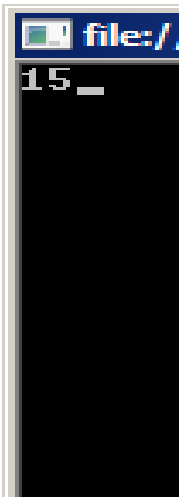
في حالة الرغبة في معرفة العدد الفعلي لعناصر المصفوفة نقوم بطرح طول المصفوفة من واحد

```
Sub Main()

    Dim k(15) As Double
    k(1) = 100
    k(2) = 300
    k(3) = 250
    k(4) = 200
    k(5) = 800

    Dim D As Double

    D = k.Length - 1
    Console.Write(D)
    Console.ReadKey()
```



بعد ذلك نقوم بإنشاء الدالة Avg الخاص بمعرفة المتوسط لمجموع قيم عناصر المصفوفة والفرق ما بين هذه الدالة والدوال الأخرى ان المعامل الذى تأخذه الدالة يكون مصفوفة والاختلاف الوحيد ما بين هذه المعاملات والمعاملات الأخرى هو انها تنتهي بقوسين بجانب اسم المعامل للدلالة علي انها من النوع المصفوفة وهنا في هذا المثال يكون اسم المعامل هو Ar ( )

وتقوم الدالة بعد ذلك بحساب المتوسط عن طريق قسمة مجموع العناصر علي عددها وتعود بهذه القيمة في المتغير i

```
Module Module1
    Sub Main()

        Dim k(5) As Double
        k(1) = 100
        k(2) = 300
        k(3) = 250
        k(4) = 200
        k(5) = 800

        Console.WriteLine(Avg(k))
        Console.ReadKey()

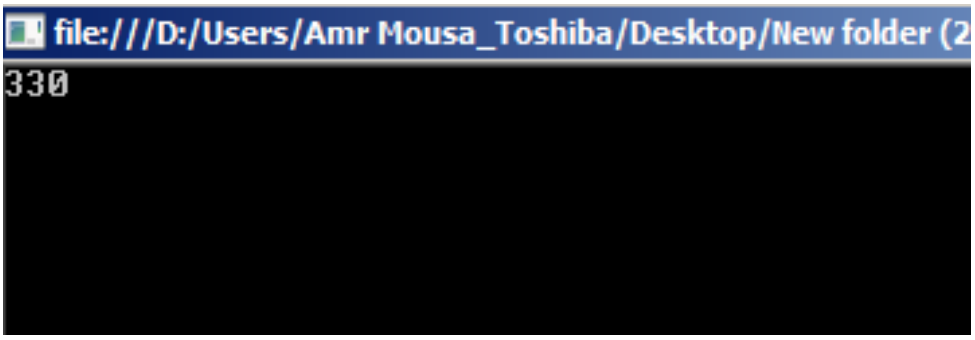
    End Sub

    Function Avg(ByVal Ar() As Double) As Double
        Dim i As Double
        i = Ar.Sum() / (Ar.Length - 1)
        Return i

    End Function

End Module
```

وعند الطباعة تظهر قيمة المتوسط بالشكل التالي:



٣٧- ادارة الاستثناءات او الاخطاء Exceptions



المقصود هنا بكلمة Exceptions هي مجموعات الاخطاء Errors التي تظهر اثناء تشغيل البرنامج وهذه الاخطاء قد تكون ناتجة عن كتابة كود خاطئ او بسبب ادخال بيانات خاطئة او القيام بعمليات خطأ وفي جميع تلك الاحوال يظهر الاعتراض او رسالة الخطأ ويقوم الفيجوال بيزك بإظهار تلك الرسائل والتي قد تختلف بحسب نوع الخطأ الذى واجه البرنامج وايضاً يظهر مكان حدوث هذا الخطأ.

فمثلاً في المثال التالي يقوم البرنامج بحساب عملية القسمة ما بين رقمين صحيحين والخطأ المتعمد في هذا البرنامج هو اننا نقوم بقسمة رقم الصحيح وهو في هذه الحالة ١٠٠٠ علي الصفر وتلك عملية رياضية خاطئة بسبب انه لا يجوز القسمة علي صفر

```
Module Module1

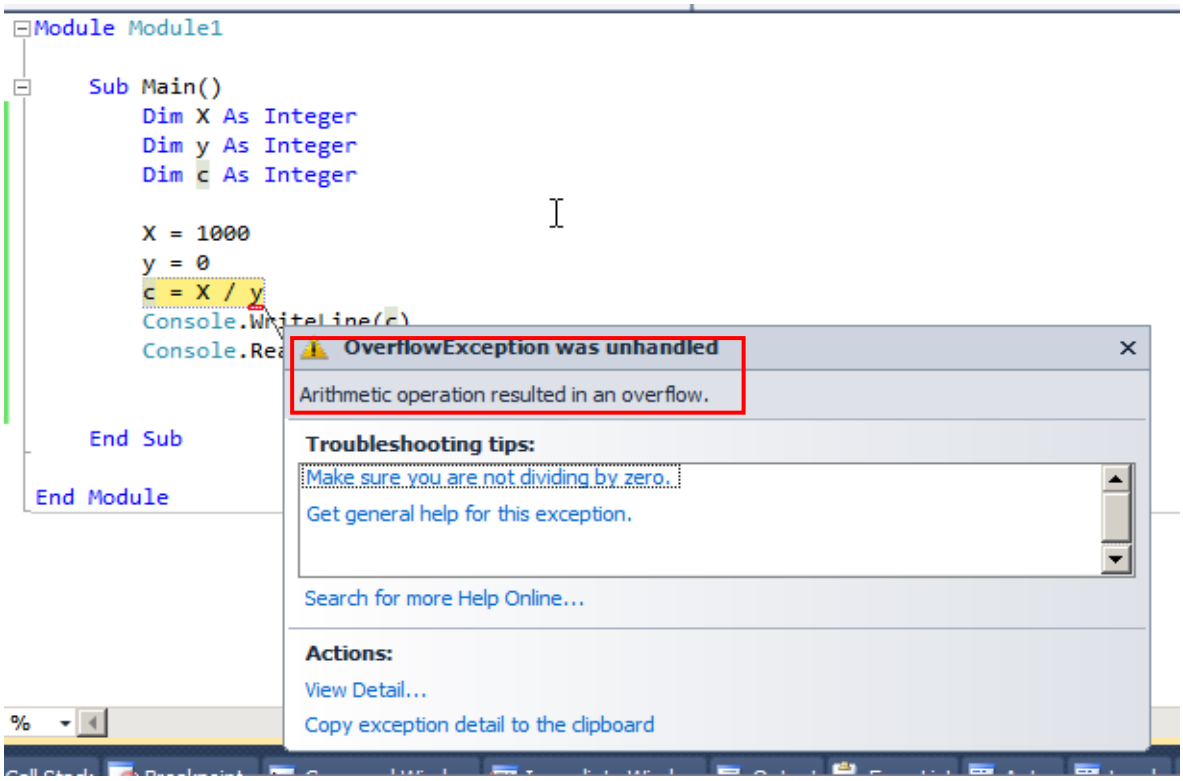
    Sub Main()
        Dim X As Integer
        Dim y As Integer
        Dim c As Integer

        X = 1000
        y = 0
        c = X / y
        Console.WriteLine(c)
        Console.ReadKey()

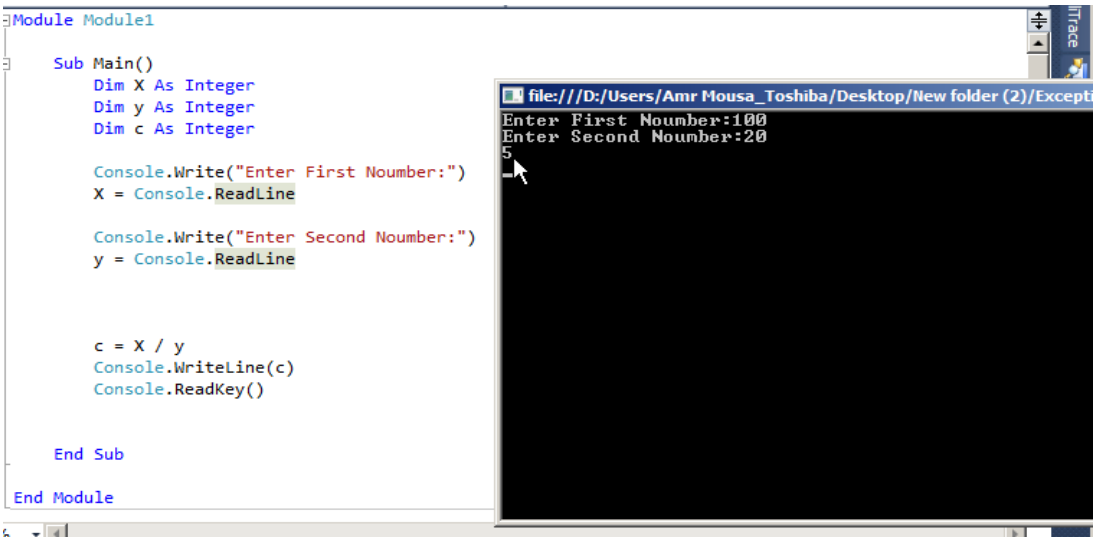
    End Sub

End Module
```

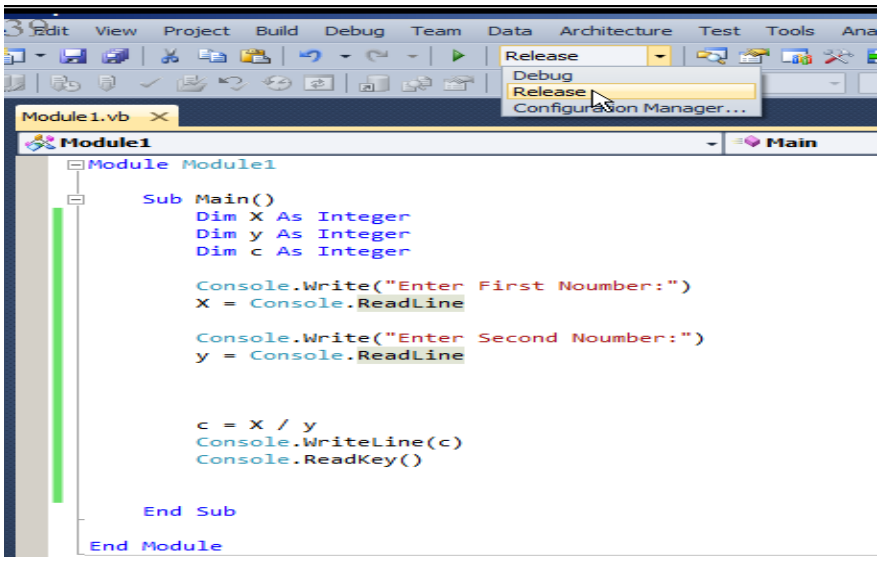
وفي تلك الحالة يظهر البرنامج رسالة الخطأ والتي تظهر بالشكل التالي ويكون نص الرسالة في تلك الحالة OverFlowException was unhanded



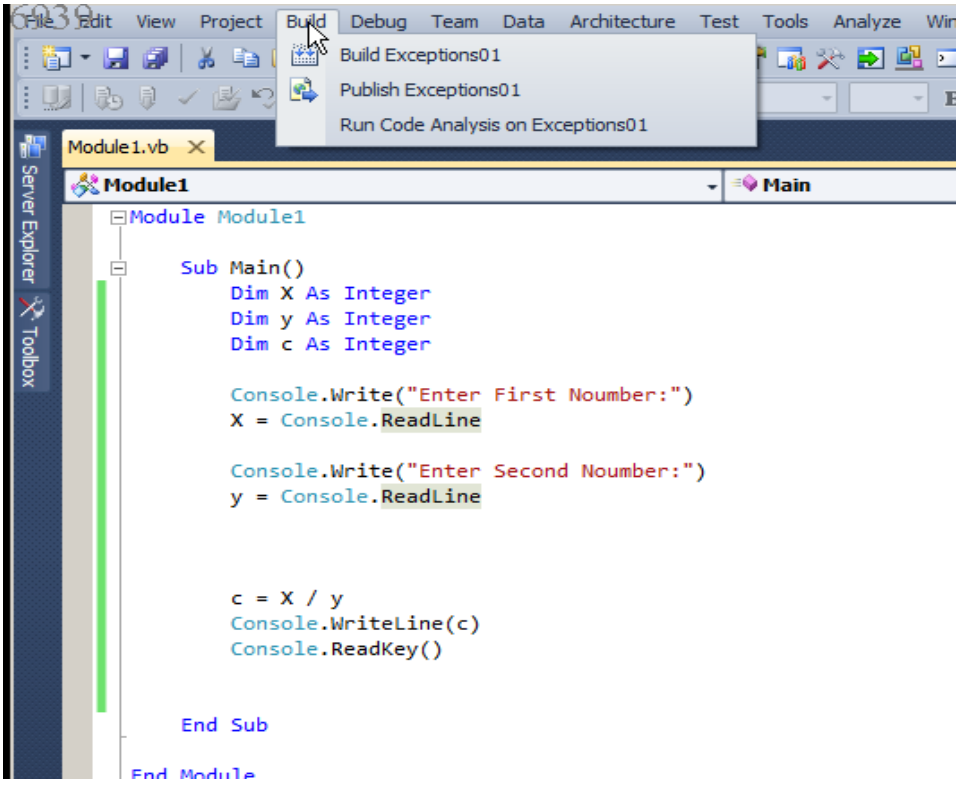
وتلك الأخطاء قد لا تكون ناتجة عن كتابة اكواد خطاء ولكنها قد تكون ناتجة عن ادخال المستخدم قيمة خطأ فمثلاً في المثال التالي قمنا بطلب ان يدخل المستخدم قيمة الرقمين الصحيحين الذى نود قسمتهما وفي حالة ادخال المستخدم قيمة صحيحة يعمل البرنامج بشكل سليم ويظهر النتيجة الصحيحة دون ان يظهر خطأ.



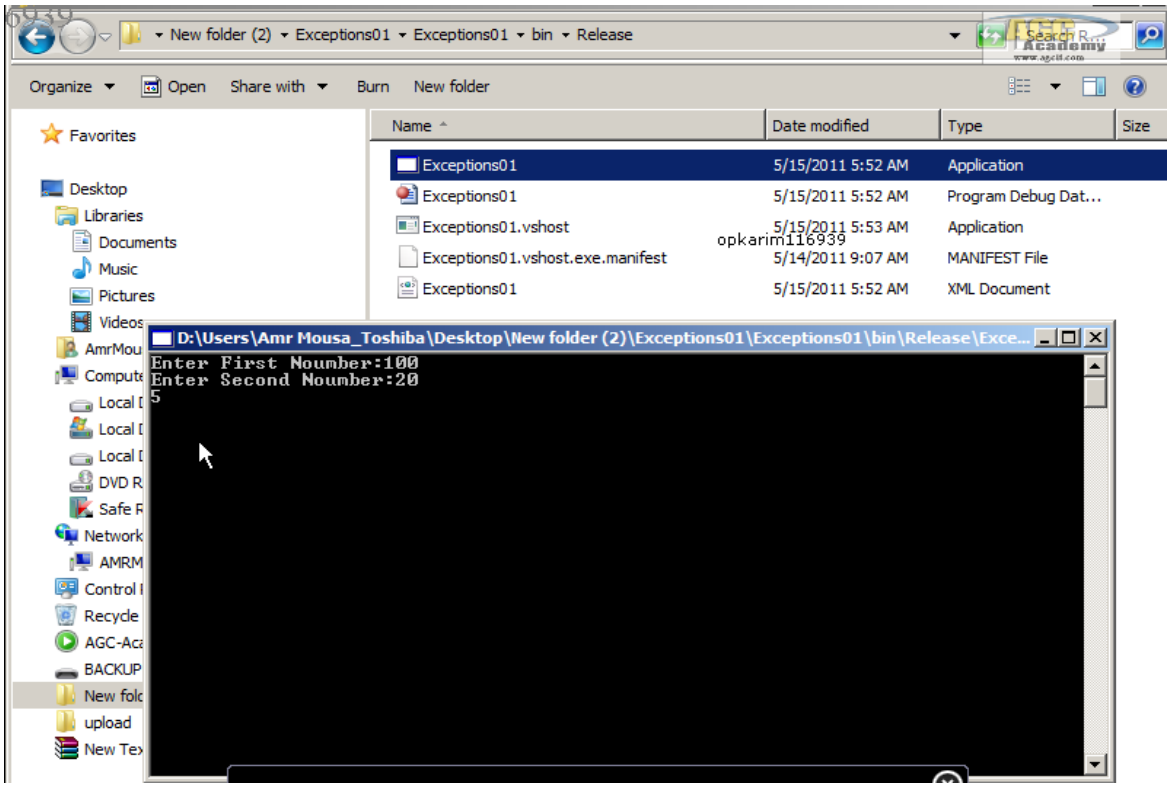
ولكن مثلاً في حالة اذا قمنا بإرسال هذا البرنامج الي مستخدم عن طريق الملف التنفيذ للبرنامج فمثلاً نقوم بعمل الملف التنفيذي للبرنامج السابق عن طريق الاختيار Release من القائمة كما في الشكل التالي:



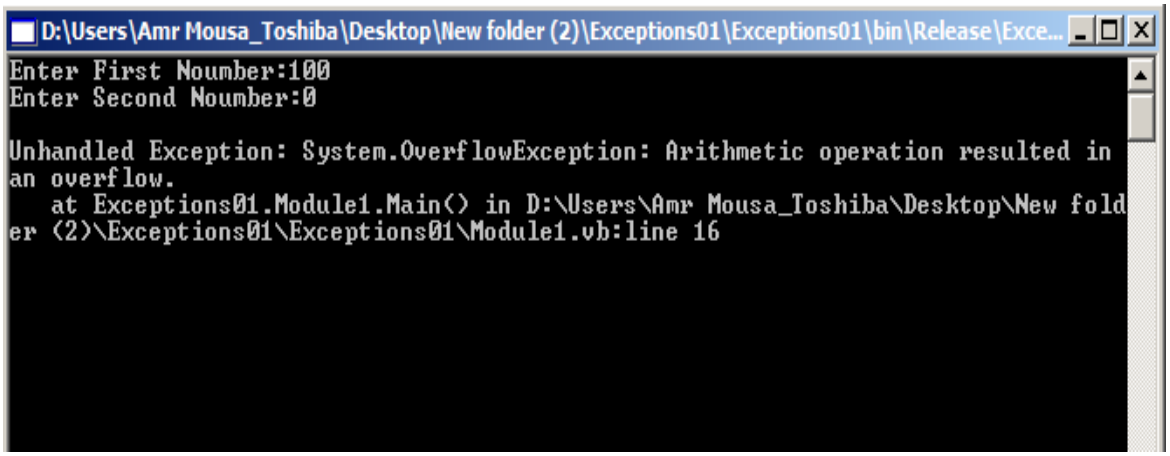
بعد ذلك نختار من القائمة Build → Build Exceptions01



بعد ذلك لنقوم بتشغيل البرنامج من الملف التنفيذي الخاص به عن طريق الذهاب الي مكان الملف التنفيذي في داخل مجلد المشروع وهو يكون في المسار bin→Release بعد ذلك نقوم بتشغيل الملف التنفيذي بالضغط عليه دبل كليك ونقوم بإدخال القيمة ١٠٠ للرقم الاول و ٢٠ للرقم الثاني فيقوم البرنامج بتنفيذ عملية القسمة بشكل سليم.



ولكن اذا افترضنا اننا ادخلنا قيمة خطأ مثل ١٠٠ وصفر فيظهر لنا الشاشة رسالة خطأ مشابهة لرسال الخطأ التي كانت تظهر اثناء التصميم بداخل الفيجوال بيزك.



وعملية التحكم في هذه الأخطاء تتم عن طريق الامر Try والامر Catch وهذين الامرين عن طريقهما نقوم بالتحكم في الأخطاء التي قد نشك في إمكانية حدوثها فمثلا في المثال السابق من الممكن ان نقوم بتصميم البرنامج بحيث انه في حالة ظهور الخطأ يقوم البرنامج بإظهار رسالة الي المستخدم تفيد حدوث خطأ في القيم التي قام بإدخالها للأرقام ويقوم بعد ذلك بسؤال المستخدم عن رغبته في اعادة المحاولة او الخروج من البرنامج.

وهذه العملية تتم عن طريق اولاً استخدام الامر Try وهذا الامر يشبه الامر if من حيث انه يقوم باختبار الاكواد التي تتم بعده وفي حالة حدوث خطأ اثناء تنفيذ هذه الاكواد نستطيع ان نقوم بالتحكم في هذا الخطأ عن طريق الامر Catch الذى يقوم بالتنفيذ في حالة ظهور خطأ.

وهنا في حالة ظهور الخطأ او ما يطلق عليه تفجير الخطأ يظهر رسالة للمستخدم تفيد ان الرقم خطأ وان يختار ما بين انهاء البرنامج او العودة للمحاولة مرة اخري وفي حالة الاختيار العودة مرة اخري ينتقل البرنامج مرة اخري الي السطر الاول الخاص بإدخال الارقام عن طريق جملة 10 GoTo فيعاود البرنامج التنفيذ مرة اخري

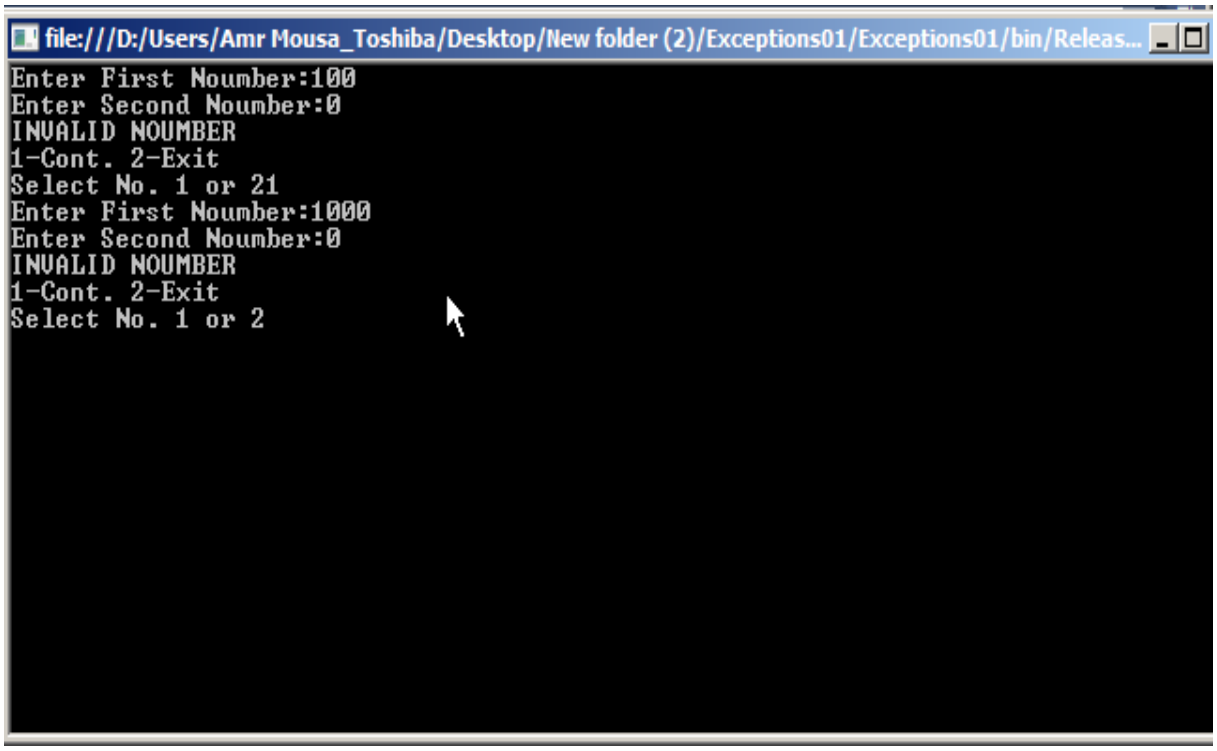
```
Module Module1
    Sub Main()
        Try

            Dim X As Integer
            Dim y As Integer
            Dim c As Integer
10:
            Console.Write("Enter First Noumber:")
            X = Console.ReadLine
            Console.Write("Enter Second Noumber:")
            y = Console.ReadLine
            c = X / y
            Console.WriteLine(c)
            Console.ReadKey()

        Catch ex As Exception
            Console.WriteLine("INVALID NOUMBER")
            Console.WriteLine("1-Cont. 2-Exit")
            Console.Write("Select No. 1 or 2")
            Dim H As Integer
            H = Console.ReadLine

            If H = 1 Then GoTo 10
            End
        End Try
    End Sub
```

وعند بدء تشغيل البرنامج في حالة ادخال قيم خطأ مثل ١٠٠ و صفر يظهر الخطأ ويعاود البرنامج التنفيذ حتى يتم ادخال القيم الصحيحة كما يظهر في المثال التالي:



٣٨- طباعة رسالة الخطأ او رقمه:

يمكن طباعة رسالة الخطأ التي تظهر عن طريق الامر ex.Message

يمكن طباعة رقم الخطأ عن طريق الامر Err.Number كما في المثال التالي:

```
Module1.vb*
Module1
                Dim X As Integer
                Dim y As Integer
                Dim c As Integer
10:
                Console.Write("Enter First Noumber:")
                X = Console.ReadLine
                Console.Write("Enter Second Noumber:")
                y = Console.ReadLine
                c = X / y
                Console.WriteLine(c)
                Console.ReadKey()

            Catch ex As Exception

                Console.WriteLine(Err.Number)
                Console.WriteLine(ex.Message)

                Console.WriteLine("1-Cont. 2-Exit")
                Console.Write("Select No. 1 or 2")
                Dim H As Integer
                H = Console.ReadLine

                If H = 1 Then GoTo 10
                End
100 %
```

في هذه الحالة رقم الخطأ هو 6

والرسالة التي تظهر هي : Arithmetic operation resulted in an overflow

```
file:///D:/Users/Amr Mousa_Toshiba/Desktop/New folder (2)/Ex
Enter First Noumber:100
Enter Second Noumber:0
6
Arithmetic operation resulted in an overflow.
1-Cont. 2-Exit
Select No. 1 or 2
```

ومن الممكن التعامل مباشرة مع الكلاس المسئول عن الاخطاء فبدلاً من اخذه نسخة منه كما في المثال السابق وهي ex من الممكن التعامل معه مباشرة عن طريق Err ومن اجل معرفة رقم الخطأ والرسالة الخاصة به يكون الامر Err.Number و Err.Description وتكون النتيجة بالمثل.

```
            Dim X As Integer
            Dim y As Integer
            Dim c As Integer
10:
            Console.Write("Enter First Noumber:")
            X = Console.ReadLine
            Console.Write("Enter Second Noumber:")
            y = Console.ReadLine
            c = X / y
            Console.WriteLine(c)
            Console.ReadKey()

        Catch ex As Exception

            Console.WriteLine(Err.Number)
            Console.WriteLine(Err.Description)

            Console.WriteLine("1-Cont. 2-Exit")
            Console.Write("Select No. 1 or 2")
            Dim H As Integer
            H = Console.ReadLine

            If H = 1 Then GoTo 10
            End
```

```
file:///D:/Users/Amr Mousa_Toshiba/Desktop/New folder (2)/Exception
Enter First Noumber:100
Enter Second Noumber:0
6
Arithmetic operation resulted in an overflow.
1-Cont. 2-Exit
Select No. 1 or 2
```

ومن الممكن ان اقوم متعمداً بتفجير او اطلاق الخطأ الذى اريده عن طريق الامر Err.Raise( ) وهنا في هذا المثال قمنا بإطلاق الخطأ رقم ٦

```
            Dim X As Integer
            Dim y As Integer
            Dim c As Integer
10:
            Console.Write("Enter First Noumber:")
            X = Console.ReadLine
            Console.Write("Enter Second Noumber:")
            y = Console.ReadLine

            If y = 5 Then Err.Raise(6)

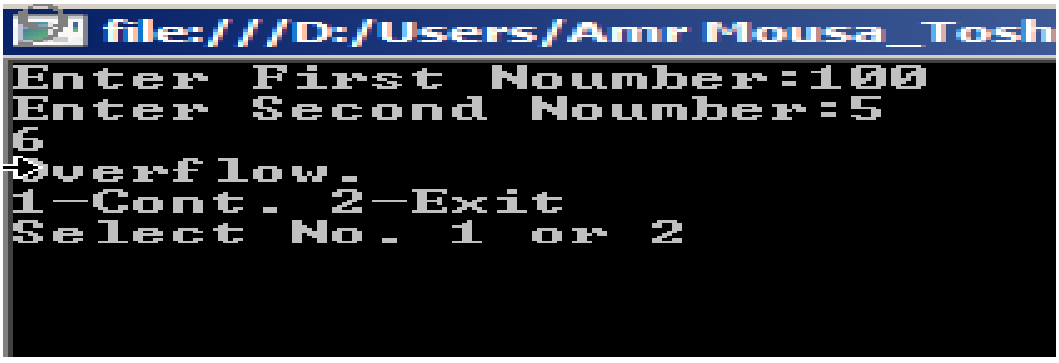

            c = X / y
            Console.WriteLine(c)
            Console.ReadKey()

        Catch ex As Exception

            Console.WriteLine(Err.Number)
            Console.WriteLine(Err.Description)

            Console.WriteLine("1-Cont. 2-Exit")
            Console.Write("Select No. 1 or 2")
```

وقمنا بعد ذلك بطباعة الرسالة الخاصة به



ومن الممكن اختيار رقم خطأ مختلف ففي المثال القادم قمنا باختيار الخطأ رقم ٧

```
Module1                                                    Main
            Console.Write("Enter First Noumber:")
            X = Console.ReadLine
            Console.Write("Enter Second Noumber:")
            y = Console.ReadLine

            If y = 5 Then Err.Raise(7)


            c = X / y
            Console.WriteLine(c)
            Console.ReadKey()

        Catch ex As Exception

            Console.WriteLine(Err.Number)
            Console.WriteLine(Err.Description)

            Console.WriteLine("1-Cont. 2-Exit")
            Console.Write("Select No. 1 or 2")
            Dim H As Integer
            H = Console.ReadLine

            If H = 1 Then GoTo 10
            End
```

وكانت النتيجة هي ان رسالة الخطأ قد اختلفت فأصبحت Out of memory

```
file:///D:/Users/Amr Mousa_Toshiba/Deskto
Enter First Noumber:100
Enter Second Noumber:5
7
Out of memory.
1-Cont  2-Exit
Select No. 1 or 2
```

٣٩ – تعمد اطلاق الأخطاء:

في بعض الاوقات قد يكون من الافضل ان نقوم بتعمد اظهار احد رسائل الخطأ من اجل التحكم في البرنامج فمثلاً اذا افترضنا اننا نقوم بتصميم برنامج يقوم بسؤال المستخدم عن كلمة السر وان يكون له فقط ثلاثة محاولات من اجل ادخال كلمة السر قد يتم استخدام كلمة End من اجل اختراق البرنامج فيكون من الافضل ان نقوم بإلغاء ظهور كلمة End واستعاضة عنها بأحد رسائل الخطأ في تلك الحالة سيتوقف البرنامج عن العمل وستظهر رسالة الخطأ بشكل لا يوضح ان البرنامج قد توقف عن العمل فنقوم بحماية البيانات التي يتداولها البرنامج.

ففي المثال التالي نقوم بسؤال المستخدم عن كلمة السر وفي حالة اذا كانت كلمة السر مختلفة عن 123 فإن البرنامج يعاود سؤاله مرة اخري عن ادخالها وله في ذلك ٣ محاولات في حالة ادخاله كلمة السر خطأ في الثلاث محاولات يقوم البرنامج بالتوقف وإرسال رسالة خطأ لا تدل علي ان سبب توقف البرنامج هو ادخال خاطئ لكلمة السر .

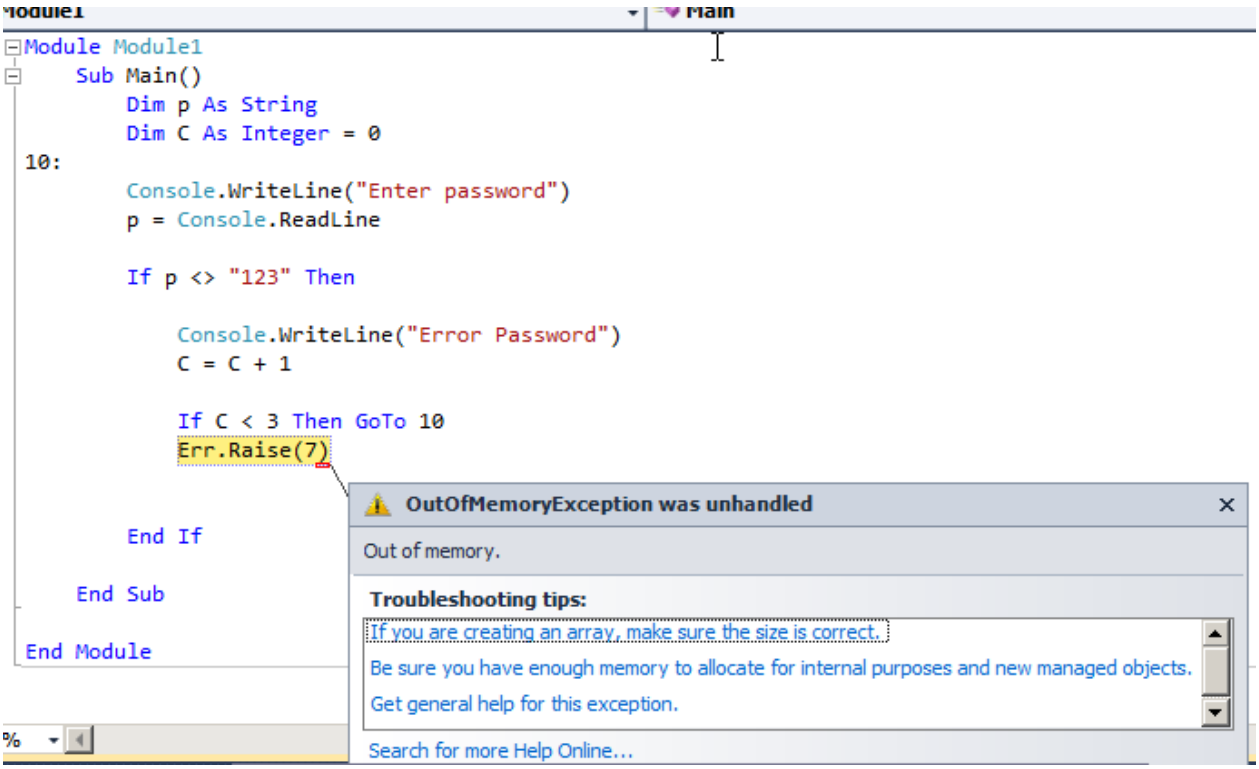
```
Module Module1
    Sub Main()
        Dim p As String
        Dim C As Integer = 0
10:
        Console.Write("Enter password")
        p = Console.ReadLine

        If p <> "123" Then
            Console.WriteLine("Error Password")
            C = C + 1
            If C < 3 Then GoTo 10
            Err.Raise(7)
        End If
        Console.WriteLine("Welcome TO AGC Program")
        Console.ReadKey()

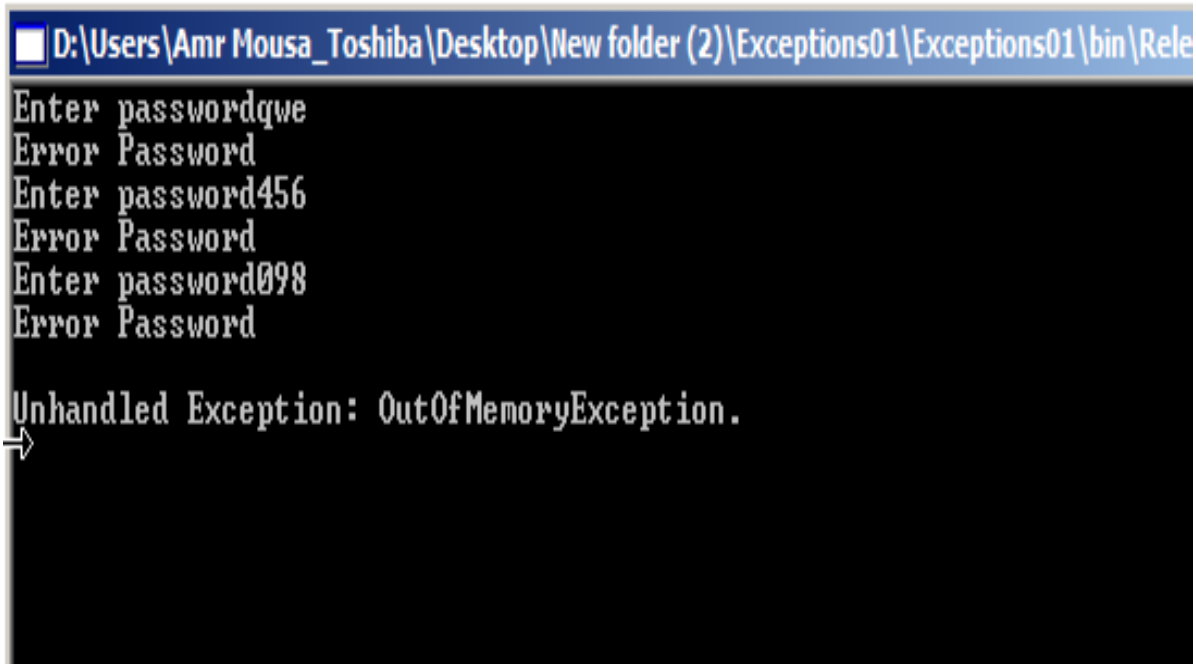

    End Sub

End Module
```

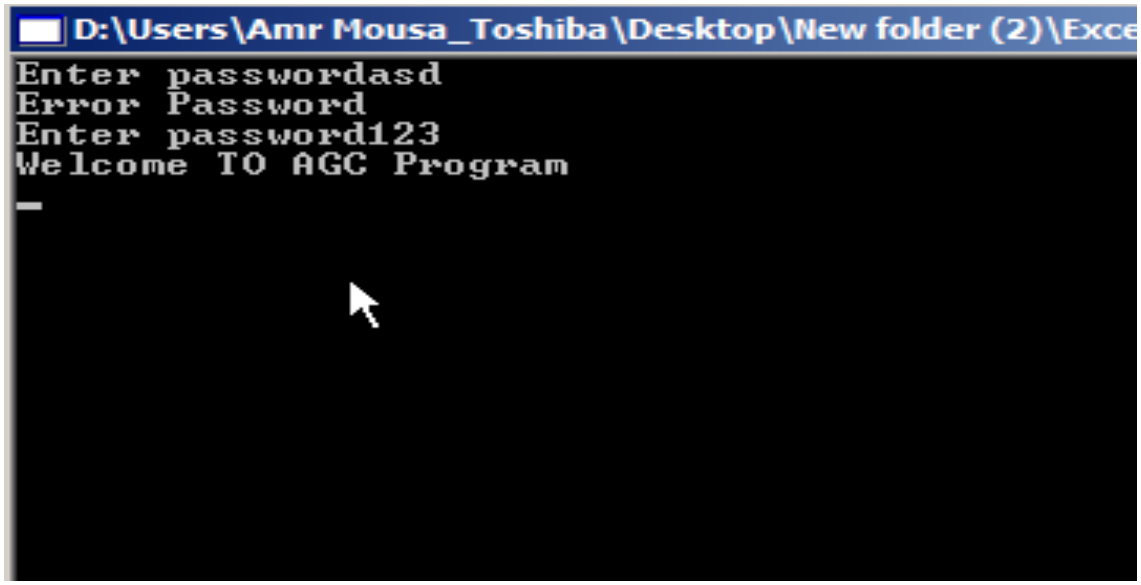
ورسالة الخطأ التي تظهر ويكون رقمها ٧ هي رسال تفيد عدم وجود فائض في الذاكرة



وفي حالة تشغيل البرنامج من الملف التنفيذي الخاص به ومحاولة ادخال كلمة السر خطأ لأكثر من ٣ مرات ستظهر الرسالة التي قمنا بإطلاقها ويتوقف البرنامج دون ان يعرف المستخدم أن السبب في ذلك هو عدد المحاولات الخطأ لكلمة السر.



وفي حالة ادخال كلمة السر صحيحة تظهر الرسال المطلوبة.



والكود التالي يقوم بإظهار مجموعات مختلفة من الرسائل الخطأ التي يمكن اظهارها او طلاقها في البرنامج الخاص بنا ويتم طباعة رسالة الخطأ والرقم الخاص بها

```
Module1                                          Main
Module Module1
    Sub Main()

        Dim i As Integer = 0

        On Error Resume Next

        For i = 1 To 100

            Err.Raise(i)
            Console.WriteLine(Err.Description & Space(10) & Err.Number)

            Err.Clear()

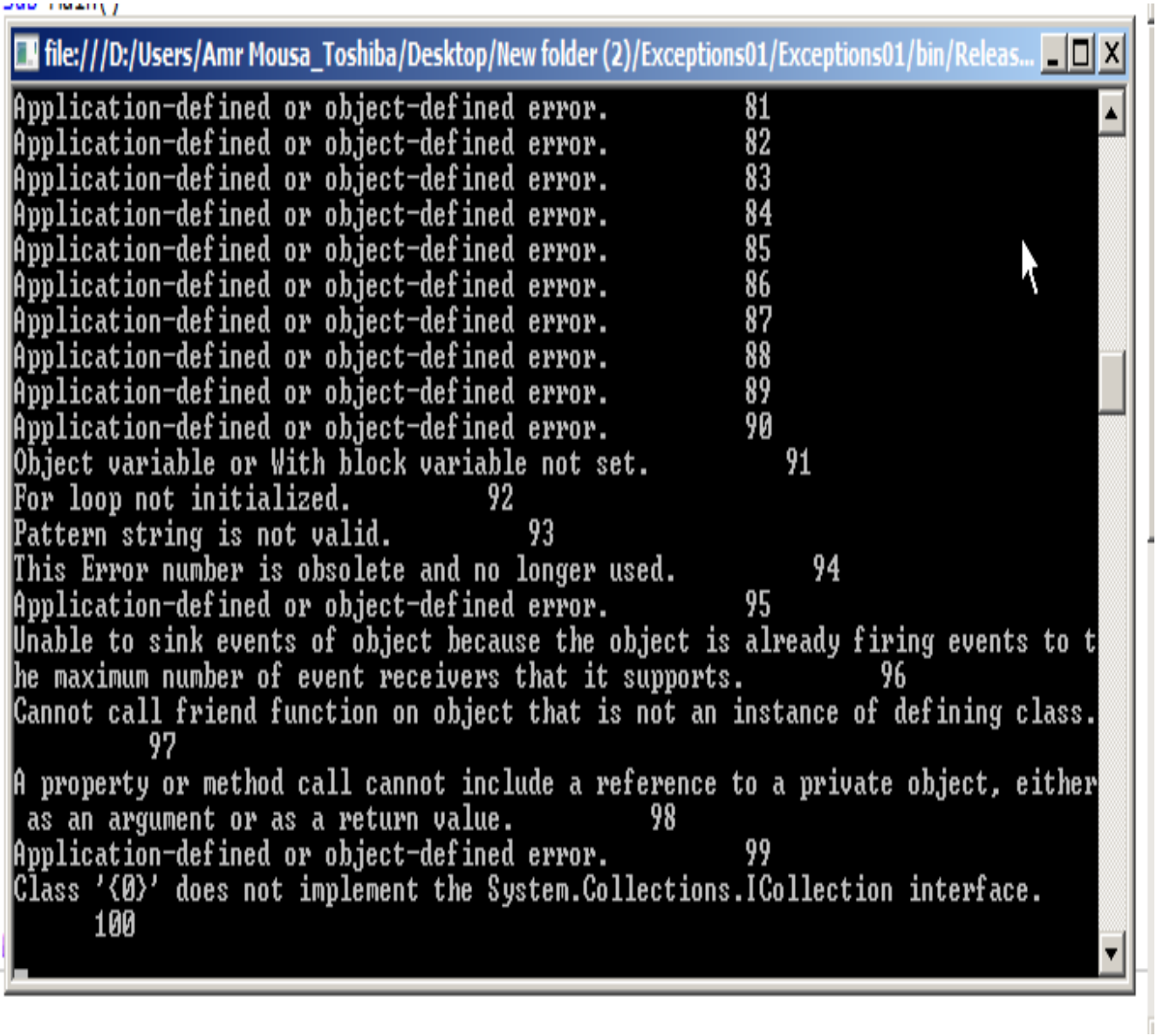
        Next

        Console.ReadKey()

    End Sub


End Module
```

وبعض هذه الارقام قد لا يكون بجانبها رسالة خطأ وهذا يدل علي عدم وجود رسالة مصاحبة لهذا الرقم



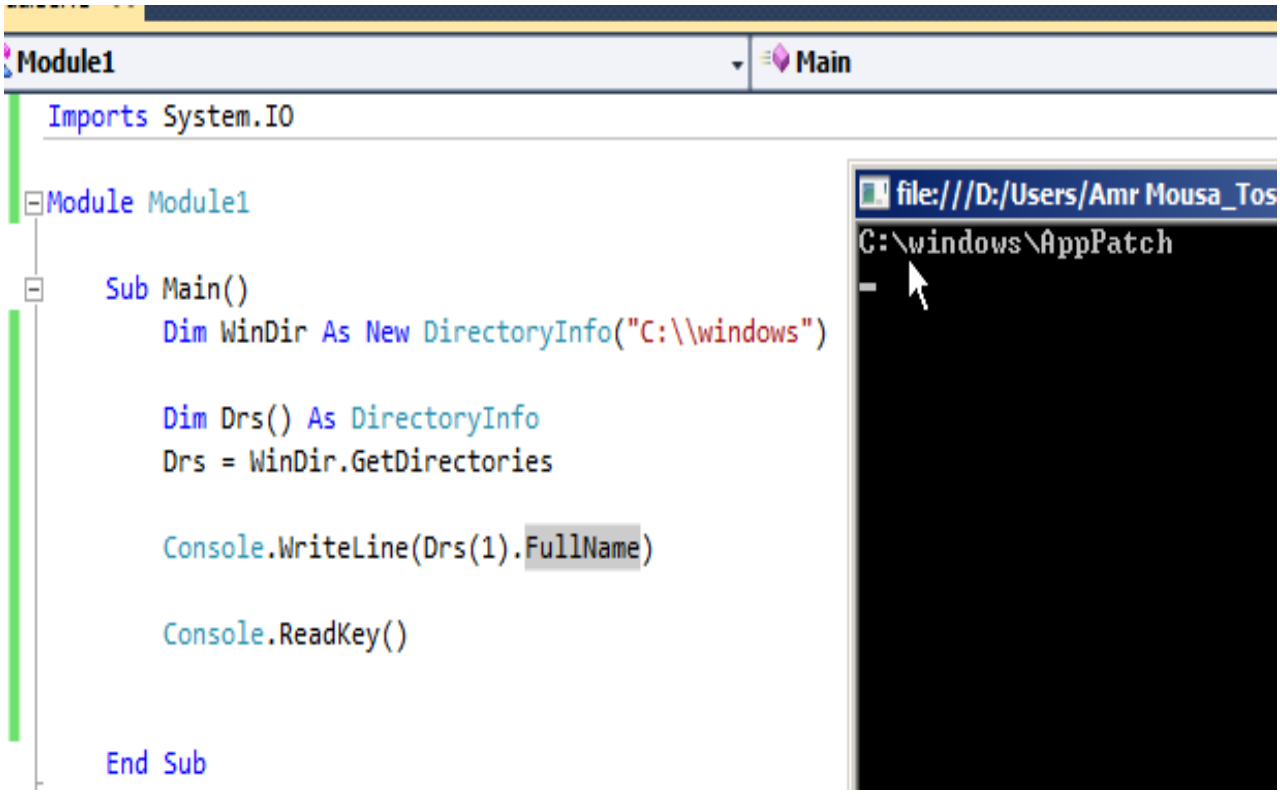
٤٠ – عرض اسماء المجلدات:

من اجل بدء التعامل مع اطار عم الدوت نت لابد اولاً من استدعاءه وذلك عن طريق الامر Import System.IO وهذا الامر يقوم باستدعاء Name Space الخاص بالتعامل مع الملفات او المجلدات الموجودة علي الجهاز وهو جزء من اطار عمل الدوت نت .

في المثال التالي قمنا بأخذ نسخة من الكلاس DirectoyInfo المسئول عن معرفة بيانات المجلدات وقمنا بإرسال مسار المجلد الذى نود معرفة البيانات الخاصة به.

بعد ذلك قمنا بأخذ نسخة من المصفوفة التي بداخلها بيانات المجلدات الموجودة بداخل الكلاس عن طريق الامر Dim Drs( ) As DirectoryInfo وقمنا بتخصيص العناصر التي ستكون بداخل المصفوفة عن طريق الامر

Drs = WinDir.GetDirectories وهنا قمنا بإدخال بيانات المجلدات في المصفوفة Drs وفي حالة الرغبة في التعامل مع احد المجلدات نقوم بذكر الرقم الخاص به وفي المثال التالي نقوم بطباعة اسم المجلد والمسار الخاص به عن طريق الدالة FullName



من اجل طباعة فقط اسم المجلد نختار البرنامج الفرعي Name

```
Module1                                                    Main
Imports System.IO

Module Module1

    Sub Main()
        Dim WinDir As New DirectoryInfo("C:\\windows")

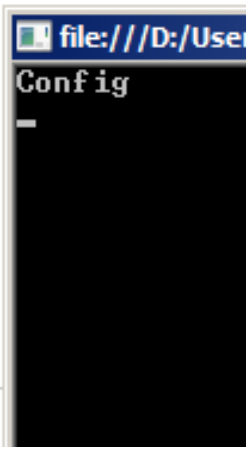
        Dim Drs() As DirectoryInfo
        Drs = WinDir.GetDirectories

        Console.WriteLine(Drs(2).Name)

        Console.ReadKey()


    End Sub

End Module
```



من اجل طباعة اسماء جميع المجلدات نقوم بعمل اللوب التالي الذى يمر علي جميع العناصر الموجودة بداخل المصفوفة Drs والتي بها المجلدات الموجودة بداخل المسار C:\\Windows

```
Imports System.IO

Module Module1

    Sub Main()
        Dim WinDir As New DirectoryInfo("C:\\windows")

        Dim Drs() As DirectoryInfo
        Drs = WinDir.GetDirectories

        Dim i As Integer = 0
        For i = 0 To Drs.Length - 1

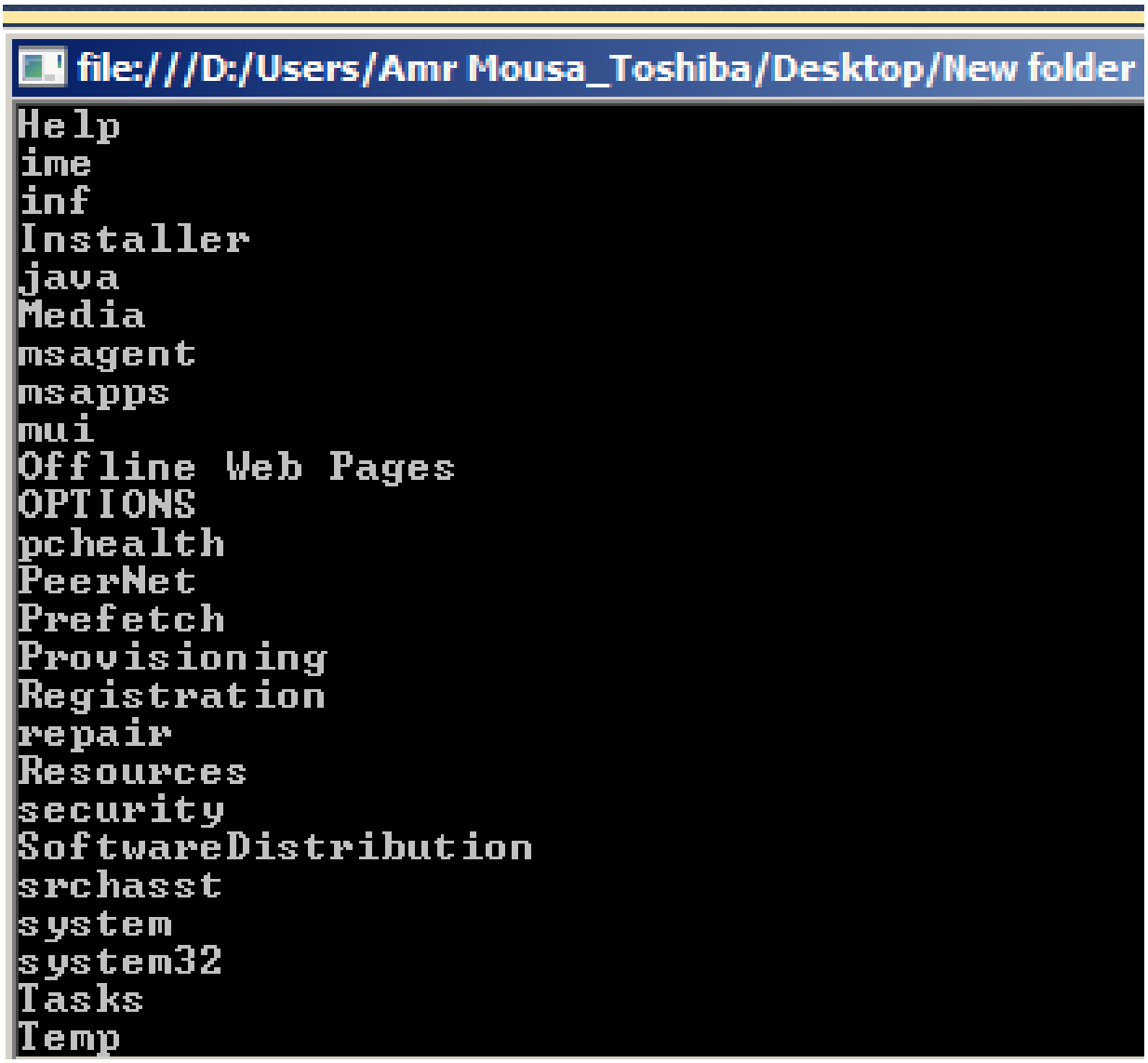
            Console.WriteLine(Drs(i).Name)
        Next

        Console.ReadKey()


    End Sub

End Module
```

تظهر جميع البيانات بالشكل التالي:



٤١ – اخر وقت الدخول علي المجلد:

لمعرفة اخر توقيت تم فيه الدخول والتعامل مع المجلد نقوم باستخدام الامر LastAccesTime وهنا بالطبع لابد وان نقوم بذكر المجلد عن طريق المصفوفة التي قمنا بإنشائها وبها بيانات المجلدات. ففي المثال التالي قمنا بإنشاء حلقة تكون مهمتها طباعة اسم المجلد واخر توقيت تم التعديل فيه علي المجلد.

```
Imports System.IO

Module Module1

    Sub Main()
        Dim WinDir As New DirectoryInfo("C:\\")

        Dim Drs() As DirectoryInfo
        Drs = WinDir.GetDirectories

        Dim i As Integer = 0
        For i = 0 To Drs.Length - 1

            Console.WriteLine("{0}----->{1}", Drs(i).Name, Drs(i).LastAccessTime)
            Console.WriteLine()
        Next

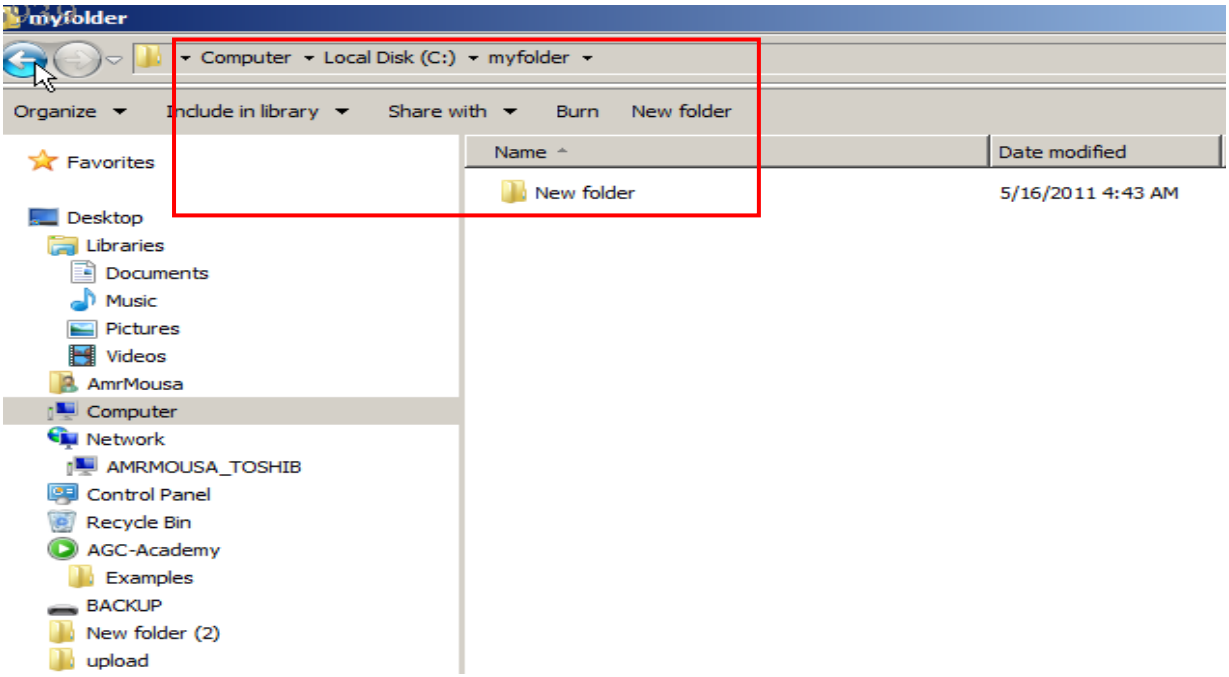
        Console.ReadKey()


    End Sub
```

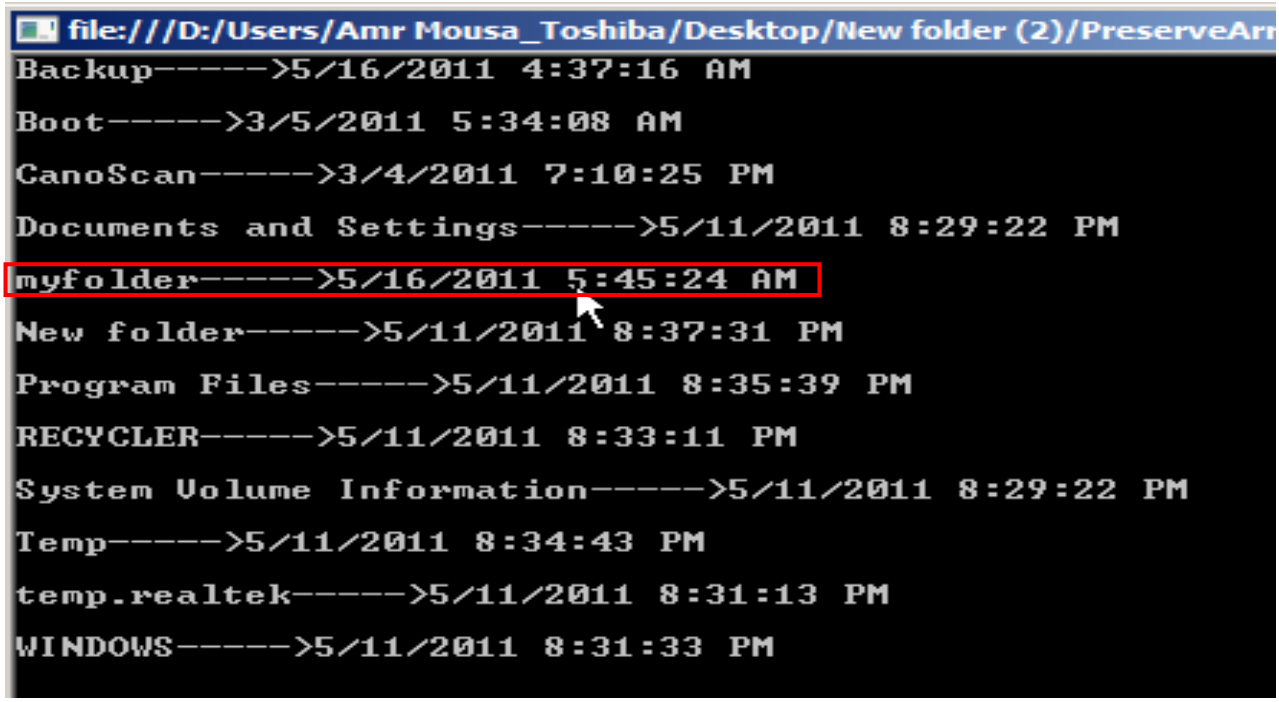
وتظهر البيانات بالشكل التالي ونلاحظ معاً توقيت التعديل علي المجلد myfolder

```
file:///D:/Users/Amr Mousa_Toshiba/Desktop/New folder (2)/PreserveArray/
Backup------>5/16/2011 4:37:16 AM
Boot------>3/5/2011 5:34:08 AM
CanoScan------>3/4/2011 7:10:25 PM
Documents and Settings------>5/11/2011 8:29:22 PM
myfolder------>5/16/2011 4:35:56 AM
New folder------>5/11/2011 8:37:31 PM
Program Files------>5/11/2011 8:35:39 PM
RECYCLER------>5/11/2011 8:33:11 PM
System Volume Information------>5/11/2011 8:29:22 PM
Temp------>5/11/2011 8:34:43 PM
temp.realtek------>5/11/2011 8:31:13 PM
WINDOWS------>5/11/2011 8:31:33 PM
```

ولنقوم بعد ذلك بالدخول الى مسار المجلد وانشاء مجلد جديد بداخله كما في الصورة التالية:



بعد ذلك نقوم بإعادة تنفيذ الاكواد السابقة وسنلاحظ علي الفور انه قد حدث تغير في وقت الدخول الي المجلد myfolder وهذا دليل علي صحة الاكواد لأنه قام بذكر اخر توقيت للتعامل مع المجلد



٤٢- معرفة بيانات الملفات:

الاوامر التالية مشابهة لأوامر التعامل مع المجلدات ولكن نوع المصفوفة الخاصة ببيانات الملفات في هذه الحالة يكون من النوع FileInfo

وبالمثل لمعرفة اسم الملف نقوم باستخدام الامر Name ويسبقه رقم الملف بداخل المصفوفة كما في الشكل التالي:

```
Module1                                          Main
Imports System.IO

Module Module1

    Sub Main()
        Dim WinDir As New DirectoryInfo("C:\\Windows")

        Dim F() As FileInfo

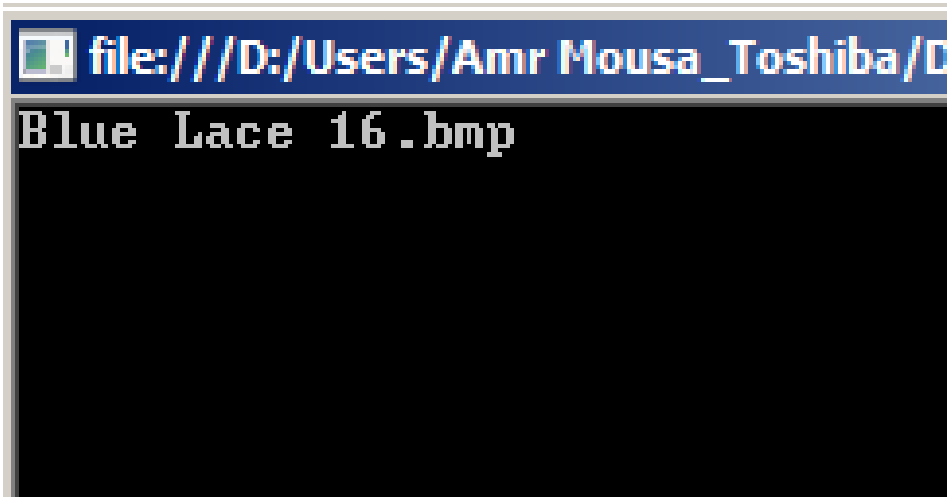
        F = WinDir.GetFiles

        Console.WriteLine(F(1).Name)
        Console.ReadKey()

    End Sub

End Module
```

ويتم طباعة اسم الملف بالإضافة الي الاسم الممتد الخاص به



في حالة الرغبة في معرفة اسم الملف بالإضافة الي المسار الموجود به الملف بداخل المجلد نقوم باستخدام الامر FullName

في الصورة التالية نقوم بطباعة اسماء ومسارات جميع الملفات الموجودة بداخل المجلد C:\\Windows

```
Imports System.IO

Module Module1

    Sub Main()
        Dim WinDir As New DirectoryInfo("C:\\Windows")

        Dim F() As FileInfo

        F = WinDir.GetFiles
        Dim i As Integer = 0

        For i = 0 To F.Length - 1

            Console.WriteLine(F(i).FullName)

        Next

        Console.ReadKey()


    End Sub

End Module
```

وتظهر بالشكل التالي وبالطبع هذه قائمة مختصرة عما يتم طباعته



في حالة الرغبة في معرفة الاسم الممتد فقط للملفات او احد الملفات نقوم باستخدام البرنامج الفرعي Extension وبالمثل نقوم في الصورة التالية بطباعة جميع الاسماء الممتد لجميع الملفات الموجود في مسار المجلد

```
Imports System.IO

Module Module1

    Sub Main()
        Dim WinDir As New DirectoryInfo("C:\\Windows")

        Dim F() As FileInfo

        F = WinDir.GetFiles
        Dim i As Integer = 0

        For i = 0 To F.Length - 1

            Console.WriteLine(F(i).Extension )

        Next

        Console.ReadKey()


    End Sub

End Module
```

وتظهر كما في الشكل التالي:



في حالة الرغبة في معرفة حجم احد الملفات فإننا نقوم باستخدام الأمر length مع ملاحظة ان الحجم الذى يتم طباعته يكون بالبايت، والاوامر التالية تقوم بطباعة احجام جميع الملفات الموجودة بداخل المسار السابق الاشارة اليه.

```
Imports System.IO
Module Module1

    Sub Main()
        Dim WinDir As New DirectoryInfo("C:\\Windows")

        Dim F() As FileInfo

        F = WinDir.GetFiles
        Dim i As Integer = 0

        For i = 0 To F.Length - 1

            Console.WriteLine(F(i).Length)

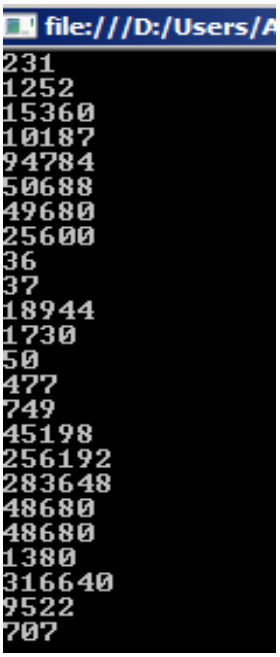
        Next

        Console.ReadKey()


    End Sub

End Module
```

وتظهر الاحجام كما اشرنا بالبايت وهي تتفاوت حسب نوع الملف والبيانات الموجودة بداخله.



٤٣- أمر التعامل مع اخطاء اثناء تنفيذ الحلقة او اللوب For Next

On Error Resume Next

```
Dim Drs() As DirectoryInfo = WinDir.GetDirectories

Dim f() As FileInfo

Dim b As Integer = 0
Dim i As Integer = 0

On Error Resume Next

For i = 0 To Drs.Length - 1
    Console.WriteLine(Drs(i).Name)
    Console.WriteLine("----------------------")

    f = Drs(i).GetFiles

    For b = 0 To f.Length - 1
        Console.WriteLine(f(b).Name)
    Next


Next
```

والكود السابق مسئول عن طباعة اسماء الملفات الموجودة بداخل المجلدات الموجودة في المسار الذى نقوم بإرساله اليها وبعض هذه الملفات قد لا يكون لدينا إمكانية الوصول الي البيانات الخاصة بها ففي تلك الحالة قد يتوقف التنفيذ عند ملف معين ولا يتم إظهار بيانات باقي الملفات في تلك الحالة يتم الخروج من الحلقة وإظهار خطأ لتفادي هذا الخطأ واكمال الحلقة نقوم باستخدام الامر

On Error Resume Next

٤٤- اغلاق الصلاحيات للملفات:

يمكن اثناء التعامل مع الملفات ان نغلق امكانية التعامل مع احد الملفات علي المستخدم الحالي للجهاز بحيث لا يمكن لأي مستخدم اخر ان يقوم بإمكانية التعديل علي تلك الملفات وذلك عن طريق الامر.

Encrypt ويمكن الغاء تلك العملية عن طريق الامر Decrypt وتلك الاوامر يتم الوصول اليها عن طريق التعامل مع كل ملف في مصفوفة بيانات الملفات.

٤٥- انشاء وحذف المجلد – تحريك الملفات بداخل المجلد

يتم انشاء المجلد عن طريق الامر Create ولكنه لابد اولاً من التأكد من ان المجلد غير موجود عن طريق الامر Exist وفي تلك الحالة يتم انشاءه بنفس الاسم والمسار الذى قمنا باختياره

ويتم حذف المجلد عن طريق الامر Delete ولكن ايضاً لابد وان يكون المجلد موجوداً مسبقاً

من اجل تغير مكان المجلد يتم نقل الملفات الموجودة بداخل هذا المجلد الي مسار مجلد اخر ويتم حذف المجلد القديم ويتم انشاء مجلد جديد في المكان الذى نقوم باختياره عن طريق الامر MoveTo

٤٦- التعامل مع مدخلات سطر الاوامر او Environment Variable

يقصد بهذا المصطلح ان البرنامج الذى تقوم بتنفيذه من سطر الاوامر من الممكن ان نقوم بإرسال بعض المدخلات اليه قبل تشغيله بحيث يتم معالجة تلك المدخلات وفي بعض البرامج الأخرى يطلق علي هذه المدخلات Optional Arguments فمثلاً اذا قمت باستدعاء البرنامج تحت بيئة سطر الاوامر فمن الطبيعي ان تلك العملية تتم عن طريق كتابة اسم الملف التنفيذ للبرنامج في تلك الحالة من الممكن بعد ان اكتب اسم البرنامج ان اتبع الاسم بمجموعة من المتغيرات او الاوامر الأخرى التي يستطيع البرنامج ان يتعامل معها من اجل معرفة تلك المتغيرات التي تم ادخالها اثناء تشغيل البرنامج نستخدم الكلاس Environment.CommandLine

ففي المثال التالي قمنا بإدخال هذه البيانات في داخل المتغير x ثم بدأ البرنامج في فحص هذه المتغيرات وفي حالة اذا ما كانت هذه المتغيرات يوجد بها كلمة AGC فانه يقوم بطباعة الرسالة المكتوبة بالأسفل علي الشاشة وفى غير ذلك يقوم بإخراج رسالة خطأ.

```
Module1                                          Main
Imports System.IO

Module Module1

    Sub Main()

        Dim X As String
        X = Environment.CommandLine

        If InStr(X, "AGC") > 0 Then
            Console.WriteLine("Welcome to AGC Academy")
            Console.ReadKey()
            End
        End If

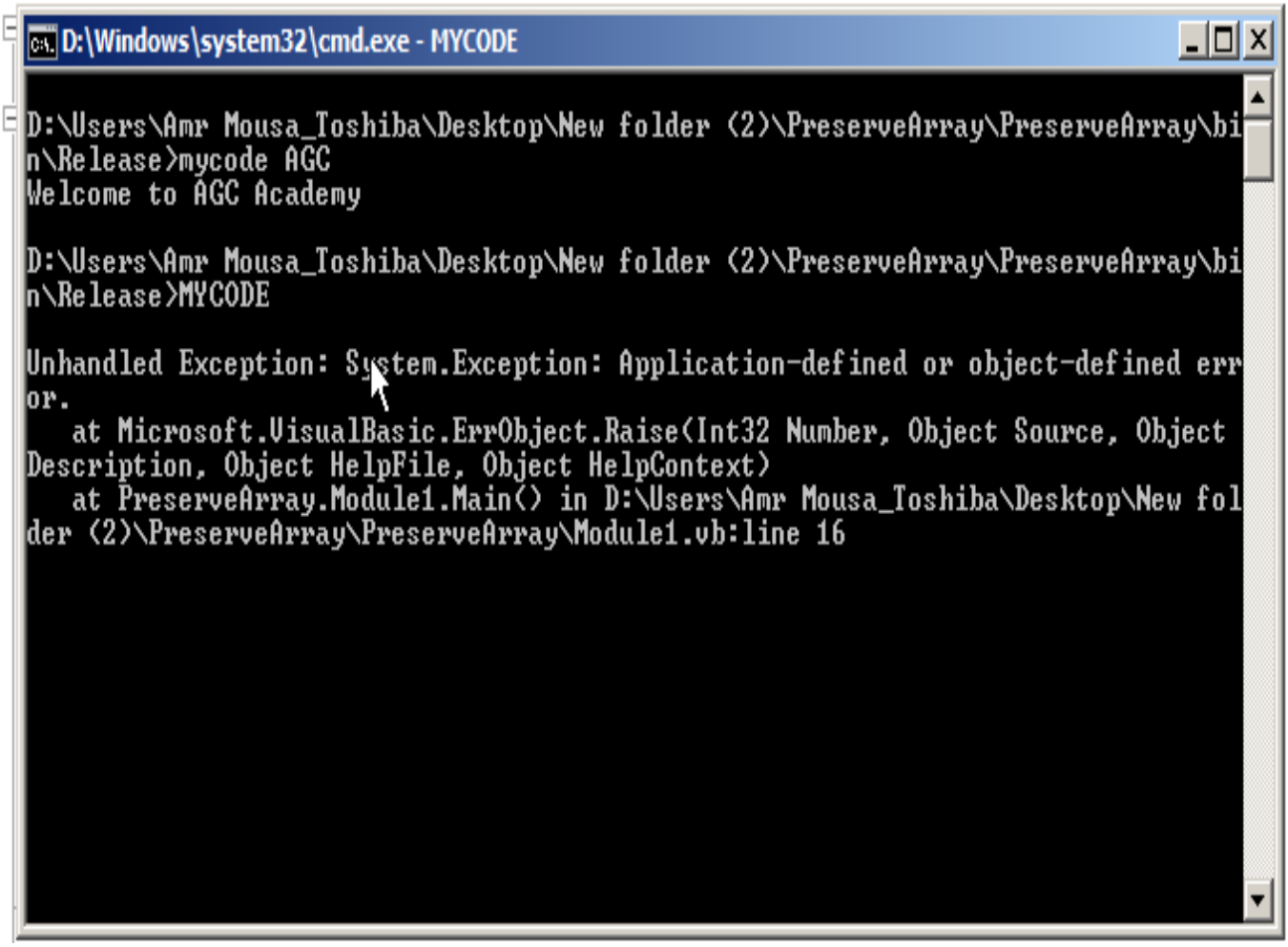
        Err.Raise(8)


        Console.ReadKey()


    End Sub

End Module
```

وعند بدء تنفيذ البرنامج في المرة الاولي قمنا بعد استدعاء اسم البرنامج بكتابة كلمة agc ففي تلك الحالة قام بطباعة الرسالة المطلوبة وفي المرة الثانية بسبب عدم كتابة الكلمة الصحيحة قام بإظهار رسالة الخطأ .



٤٧- لمعرفة المجلد الحالي الذى يتم تنفيذ البرنامج من داخله يتم استخدام الامر

CurrentDirectory ومن الممكن التعديل علي هذه القيمة بإدخال قيمة اخري مناسبة لنا.

لمعرفة حجم ذاكرة الجهاز او الرام يتم استخدام الامر WorkingSet

٤٨ – بعض اوامر التعامل مع المجلدات والملفات:

امر انشاء مجلد Mkdir( )

امر حذف مجلد Rmdir( )

امر اعادة تسمية مجلد Rename( )

امر حذف ملف Kill( )

لمعرفة المجلد الحالي الذى يعمل منه البرنامج نستخدم الامر CurDir( )

```
Imports System.IO
Module Module1

    Sub Main()
        ' MkDir("E:\\NewOne")
        ' RmDir("E:\\Newone")
        ' Rename("E:\mytextfile.txt", "E:\\myT.txt")
        ' Kill("E:\\myT.txt")
        Dim a As String = CurDir()
        Console.WriteLine(a)
        Console.ReadKey()



    End Sub

End Module
```